

Mathematical programming

A lot of real-life traffic and transportation problems can be quite simply formulated in words. In the next step, engineers typically transform such a verbal problem description into a mathematical description. Key components of the mathematical description of the problem are *variables*, *constraints*, and the *objective*. Variables are also occasionally called *unknowns*. Certain variables are under the control of the analyst. There are also variables that are not under the control of the analyst. Constraints could be physical, caused by some engineering rules, laws, or guidelines, or by a variety of financial causes. Nobody could allow more than 100 passengers for a planned flight, for example, if the capacity of the aircraft is 100 seats. This is an example of a physical constraint. Financial constraints are typically related to a range of investment decisions. For example, no one could put in road development costs of more than \$15,000,000 if the existing budget equals \$15,000,000. Variable values could be feasible or infeasible. Variable values are feasible when they satisfy all the defined constraints. An objective corresponds to the end-result which the decision maker wants to achieve by selecting a certain program of action. The typical objectives in profit-oriented organizations are revenue maximization, cost minimization, or profit maximization. Providing the best level of service (travel time, waiting time, ...) to the clients represents a common objective in a nonprofit organization.

A mathematical description of a real-world problem is called a *mathematical model* of the real-world problem. We denote the n decision variables of the studied problem by $x_j, j = 1, 2, \dots, n$. Let us assume that the observed system is subject to m constraints. The general mathematical model can be written in the following way:

Optimize

$$y = f(x_1, x_2, \dots, x_n)$$

subject to:

$$g_i(x_1, x_2, \dots, x_n) \leq a_i \quad i = 1, 2, \dots, m$$

$$x_j \geq 0 \quad j = 1, 2, \dots, n$$

where

$y = f(x_1, x_2, \dots, x_n)$ - objective function

$g_i(x_1, x_2, \dots, x_n) \leq a_i \quad i = 1, 2, \dots, m$ - constraints

$x_j \geq 0 \quad j = 1, 2, \dots, n$ - constraint (non-negativity restrictions)

Optimization looks for the best value (*optimal value*) of the objective function. Optimization frequently proposes the maximization or minimization of the objective function. *Optimal solution* to the model is the finding of a set of variable values (feasible) that produce the optimal value of the objective function.

An *algorithm* represents some quantitative method used by an analyst to solve the defined mathematical model. Algorithms are composed of a set of instructions, which are usually followed in a defined, step-by-step procedure. An algorithm produces an optimal (the best) solution to a defined model. The *optimal solution* to the model is the discovery of a set of variable values (feasible) that generate the optimal value of the objective function. Depending on a defined objective function, the optimal solution corresponds to maximum revenue, minimum cost, maximum profit, etc.

1.1 LINEAR PROGRAMMING IN TRAFFIC AND TRANSPORTATION

In a lot of cases, all the variables in a problem are continuous variables. In addition, there is typically only one objective function. Often, the objective function and all constraints are *linear*, meaning that any term is either a constant or a constant multiplied by a variable. Any mathematical model that has one objective function, all continuous variables, a linear objective function, and all linear constraints is called a *linear program* (LP). It has been shown, over many years, that many real-life problems can be formulated as linear programs. Linear programs are frequently solved using broadly spread simplex algorithms.

Example 1.1

There are two types of goods that could be transported from airport A to airport B. The unit transportation cost of the first type of goods is \$200/t, whereas the unit transportation cost of the second type of goods equals \$150/t. The total aircraft cargo capacity equals 40 t and the aircraft cargo volume equals 240 m³. The volumes per ton of goods are 8 m³/t and 4 m³/t for the first and second types of goods, respectively. Determine the quantity of the first type of goods and the quantity of the second type of goods that would maximize the air carrier's revenue.

SOLUTION:

Let us introduce the following notation:

- x_1 – the quantity of first type of goods that should be transported
- x_2 – the quantity of second type of goods that should be transported

The linear program reads:

Maximize

$$F = 200 \cdot x_1 + 160 \cdot x_2$$

subject to:

$$x_1 + x_2 \leq 40$$

$$8 \cdot x_1 + 4 \cdot x_2 \leq 240$$

$$x_1 \geq 0, x_2 \geq 0$$

This problem can be solved graphically, as we have two decision variables. Our axes are x_1 and x_2 . We take into consideration only positive values of the decision variables since $x_1 \geq 0, x_2 \geq 0$. Each constraint is represented by one straight line. We replace signs “ $\leq$ ” or “ $\geq$ ” by “ $=$ ” for each constraint. In our example, the constraints $x_1 + x_2 \leq 40$ and $8 \cdot x_1 + 4 \cdot x_2 \leq 40$ are represented by straight-line equations $x_1 + x_2 = 40$ and $8 \cdot x_1 + 5 \cdot x_2 = 240$ (Figure 1.1). These straight lines are plotted in Figure 1.1. In the next step, we determine the feasible solution space, taking into account each constraint. This task can readily be completed by taking one point at a time and checking whether that point belongs to the feasible area. For example, let us consider the point with coordinates $x_1=0$ and $x_2=0$. By substituting these values into the first constraint, we obtain: $x_1 + x_2 \leq 40 \Rightarrow 0 + 0 \leq 40$. We conclude that this point belongs to the feasible solution space of the first constraint. The directions of arrows in Figure 1.1 designate the feasible region in which each constraint is satisfied, when the inequality is made active. Each point inside or on the border line of the feasible region complies with all the constraints. Every point that satisfies all the constraints is called a feasible point. We also represent the objective function by the straight line. We set that the objective function equals the arbitrary values. For example, we can set that $F=0$, e.g. $200 \cdot x_1 + 160 \cdot x_2 = 0$. For this equation, we plot the straight line. We do the same for some other arbitrary values, and we detect the direction in which our objective function changes. The optimal solution is the point that belongs to the feasible solution space and has the highest objective function value. We note that the highest objective function value has the point with the coordinates $x_1=20$ and $x_2=20$ (one of the corner points). For these values of the decision variables, the objective function equals $F=\$7200$ (Figure 1.2).

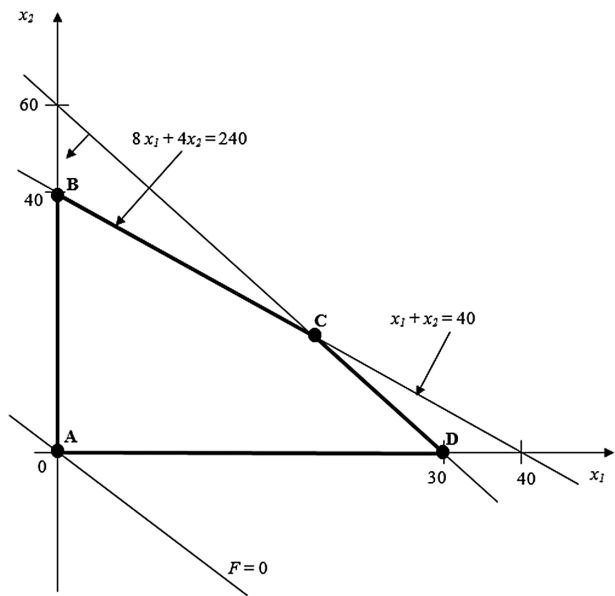


Figure 1.1 Solution space

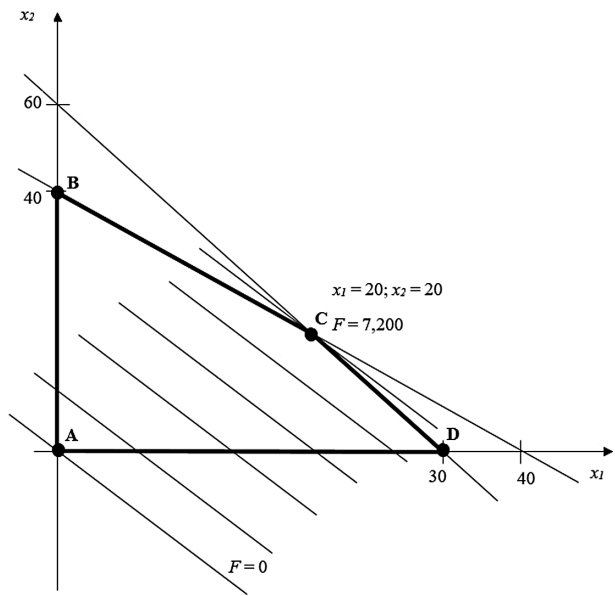


Figure 1.2 Optimal solution

We have used a graphical method for solving the given linear program. This method is suitable when there are two variables but is inconvenient for linear programs that have more decision variables. The other widely used solution approach is the simplex method.

1.2 THE SIMPLEX METHOD

Graphical solutions to many problems show that the optimal solution is associated with a corner point of the solution space every time. Unfortunately, the graphical method is appropriate only for problems that include two variables. The simplex method can be used for any number of decision variables. The simplex method is based on the idea that the optimum solution is, at all times, associated with a corner point of the solution space, as illustrated by the previous example.

The simplex method starts from one feasible corner point and improves the solution in iterations. The method moves from one corner point to an adjacent corner point. The algorithm stops when the optimal solution is reached. To demonstrate how this method works, let us consider again the previous example.

The simplex method assumes that the solution space is represented by the standard form. The standard form assumes the following: (a) the objective function is minimization or maximization; (b) all the constraints are equations; (c) all the variables are nonnegative. This means that all constraints of the type $\leq$ or $\geq$ are converted to equations. This is done by adding a slack variable or by subtracting a surplus variable to the left side of the constraint.

Let us consider our first constraint:

$$x_1 + x_2 \leq 40$$

By adding the slack variable, $x_3 \geq 0$, we convert this constraint into the following equation:

$$x_1 + x_2 + x_3 = 40$$

Similarly, by adding the slack variable $x_4 \geq 0$, we convert the constraint:

$$8 \cdot x_1 + 4 \cdot x_2 \leq 240$$

into the following equation:

$$8 \cdot x_1 + 4 \cdot x_2 + x_4 = 240$$

Let us solve the considered linear program by the simplex method. The standard form reads:

$$F - 200 \cdot x_1 - 160 \cdot x_2 = 0$$

$$x_1 + x_2 + x_3 = 40$$

$$8 \cdot x_1 + 4 \cdot x_2 + x_4 = 240$$

$$x_1 \geq 0, \quad x_2 \geq 0, \quad x_3 \geq 0, \quad x_4 \geq 0$$

where x_3 and x_4 are nonnegative slack variables.

Let us, for example, consider the case where $x_3 = 0$. This is equivalent to $x_1 + x_2 = 40$. This equation represents the edge BC in Figure 1.2. By exploring the extreme points in Figure 1.2, we conclude that every extreme point has two variables equal to zero (zero variables). It is important to note that the difference between the number of unknowns (four in our case) and the number of equations (two in our case) is also equal to two. The number of zero variables at extreme points is always equal to the difference between the number of unknowns and the number of equations.

The extreme points and the corresponding zero and nonzero variables are given in Table 1.1.

In the simplex method, the variables equal to zero are called non-basic variables. The other variables are called basic variables.

Let us insert the equations obtained into the simplex table (Table 1.2). In the leftmost column are inserted the basic variables. In this example, the basic variables are x_3 and x_4 . In this solution, their values are given in the column "Solution": $x_3 = 40$ and $x_4 = 240$. All other variables are non-basic.

Table 1.1 The extreme points, and the corresponding zero and nonzero variables

Extreme point	Zero variables	Nonzero variables
A	x_1, x_2	x_3, x_4
B	x_1, x_3	x_2, x_4
C	x_3, x_4	x_1, x_2
D	x_2, x_4	x_1, x_3

Table 1.2 Simplex table with the initial feasible solution

	F	x_1	x_2	x_3	x_4	Solution
F	1	-200	-160	0	0	0
x_3	0	1	1	1	0	40
x_4	0	8	4	0	1	240

The non-basic variables, in our example, are x_1 and x_2 and they equal zero ($x_1=0, x_2=0$). For this solution, the objective function equals 0. This is also given in the column “Solution” (in the first row).

The solution obtained is feasible if the following condition is satisfied. Let us note the basic variables. In our examples, they are x_3 and x_4 . All the elements in the columns of variables x_3 and x_4 must be zero, except for elements that are at the intersections with rows of variables x_3 and x_4 . In our example, we note that this condition is satisfied. We will demonstrate in the next example what we have to do if this condition is not satisfied.

Once again, our first feasible solution is $x_1=0, x_2=0, x_3=40$, and $x_4=240$. For this solution, the objective function value equals 0. The question in this moment is: is this solution the optimal one? The answer is no. How do we know that the solution is not optimal? We note the elements in the row of the objective function (row denoted by F). If there were no negative elements, then the solution would be optimal. In our example, two elements are negative (-200 and -160) and, because of that, this solution is not optimal (In our example, we are maximizing the objective function. Obviously, the objective function value could be improved by increasing the x_1 or x_2 to greater than zero). Generally speaking, when the solution is not optimal, it can be improved. We try to make improvements to the objective function value in the following way. One of the non-basic variables becomes the basic variable. We term the column of the non-basic variable that will become the basic variable the “the pivot column”. We select the pivot column in the following way. We select the decision variable that has the highest negative value in row F . In our example, it will be:

$$\min\{-200, -160\} = -200 \Rightarrow \text{column } x_1$$

In the next step, we must determine which one of the temporary basic variables will become non-basic. The row containing this variable is the pivot row. For this purpose, we must denote rows with positive elements in the pivot column. They are potential pivot rows. Among the potential rows, we will select the one that has the smallest value of the quotient of the appropriate element in the column “Solution” and of the element in the pivot column. In our example, it is:

$$\min\left\{\frac{40}{1}, \frac{240}{8}\right\} = \min\{40, 30\} = 30 \Rightarrow \text{row } x_4$$

The pivot element is the element at the intersection of the pivot row and the pivot column. After selecting the pivot element, we generate the new table in the following way:

- All elements in the pivot column are equal to zero, except the element which was the pivot element. The value of that element is 1.

- The new values of the elements in the pivot row are equal to the old values divided by the pivot number.
- The other elements are determined in the following way. Suppose that element a_{ij} is a pivot number, and that we want to determine the new value of the element a_{rs} . We should also note elements a_{is} and a_{rj} , that are the elements at the intersection of the pivot row and the column s , and at the intersection of the row r and the pivot column. The element at the intersection of row r and column s will have the new value

$$a_{rs} - \frac{a_{is} \cdot a_{rj}}{a_{ij}}.$$

By using these rules, we obtain the new simplex table. The next table for our example is given in Table 1.3 below. The new solution is $x_1=30$, $x_2=0$, $x_3=10$, $x_4=0$. For this solution, the objective function value is 6000.

Since we have one negative value in row F , this solution is not optimal. It can be improved, and, for that purpose, we should make the new iteration. We perform iterations until the optimal solution is reached.

The pivot column will be x_2 as only this variable has positive values in row F . The pivot row is obtained in the following way:

$$\min \left\{ \frac{10}{0.5}, \frac{30}{0.5} \right\} = \min \{20, 60\} = 20 \Rightarrow \text{pivot row is } x_3$$

The pivot element is at the intersection of row x_3 and column x_2 .

The new values for the elements of the table should be determined in the same way as in the previous step. The new table is shown in Table 1.4. The new solution is $x_1=20$, $x_2=20$, $x_3=0$, $x_4=0$, and $x_5=0$. The objective function value for this solution is 7200.

We note that there are no longer any negative elements in F row. Consequently, we conclude that this solution is the optimal solution.

Table 1.3 Simplex table with the second feasible solution

	F	x_1	x_2	x_3	x_4	<i>Solution</i>
F	1	0	-60	0	25	6000
x_3	0	0	0.5	1	-0.125	10
x_1	0	1	0.5	0	0.125	30

Table 1.4 Simplex table with the final solution of the problem

	F	x_1	x_2	x_3	x_4	<i>Solution</i>
F	1	0	0	120	15	7200
x_2	0	0	1	2	-0.25	20
x_1	0	1	0	-1	0.25	20

There are many software packages for solving linear programs. We are going to present how previous examples could be solved by Excel Solver, LPSolve IDE, or LINDO.

In the Excel sheet, we choose the cells for the decision variables, as well as one cell to calculate the objective function and to present the constraints. Figure 1.3 shows the appearance of the Excel sheet for the previous example. Let us suppose that the cells for the decision variables are A2 and B2. In cell B4, we calculate the objective function value in the following way: “=20*A2+15*B2”. In this example, we have two constraints. The parts of the constraints to the left of the sign “ $\leq$ ” should be calculated in the cells A7 and A8. In cell A7, we should write “=A2+B2”, and, in cell A8, Cell A8: “=8*A2+5*B2”. The values given at the right side of the sign are just given as the cells: C7 and C8.

When the sheet is prepared, we start the Solver (from the pallet data). In the window of the Solver, we shall:

- enter the cell of the objective function,
- choose whether we shall maximize or minimize the objective function,
- enter constraints, and
- choose the solving method.

Figure 1.4 shows how the Solver looks after entering the values related to our example. By pressing the button Solve, we obtain the solution given in Figure 1.5. We notice that $x_1=20$, $x_2=20$, and $F=7200$.

In the LPSolve IDE we write the objective function and the constraints in a similar way as we write on paper. Figure 1.6 shows the implementation of our linear program. When insertion of the objective function and

	A	B	C	D	E
1	x1	x2			
2					
3					
4	F =				
5					
6					
7	0	$\leq$	1500		
8	0	$\leq$	10000		
9					
10					

Figure 1.3 Data prepared in Excel for solving the problem

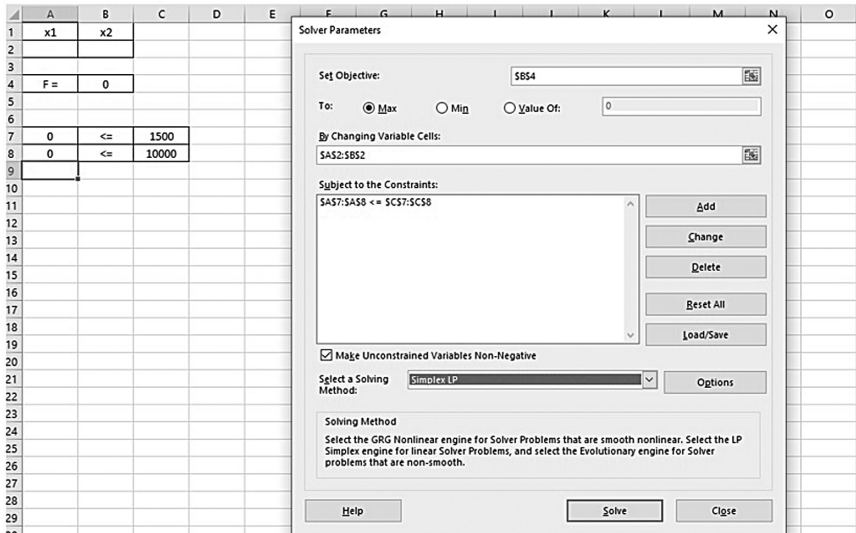


Figure 1.4 Completed Excel Solver

constraints is complete, we press the button Solve. The solution is given in Figure 1.7.

In a similar way, we can solve the linear program in LINDO. Figure 1.8 shows the implementation of our example. In the first row, we write the criteria of optimization and the objective function. After that, we write the set of constraints. Before we start with the constraints, we have to set “Subject to”, and, at the end of the set of constraints, we have to write “End”. The solution of this Solver is given in Figure 1.9.

	A	B	C	D	E
1	x1	x2			
2	20	20			
3					
4	F =	7200			
5					
6					
7	40	<=	40		
8	240	<=	240		
9					
10					

Figure 1.5 Solution obtained in Excel

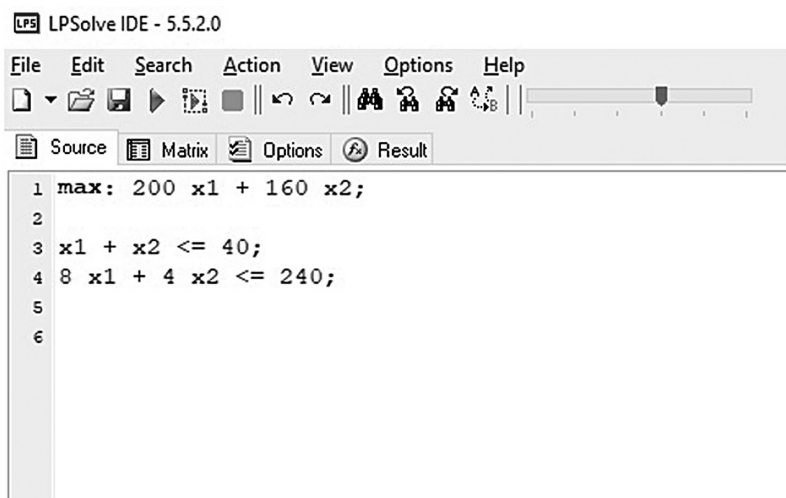


Figure 1.6 Linear program inserted in LPSolve IDE

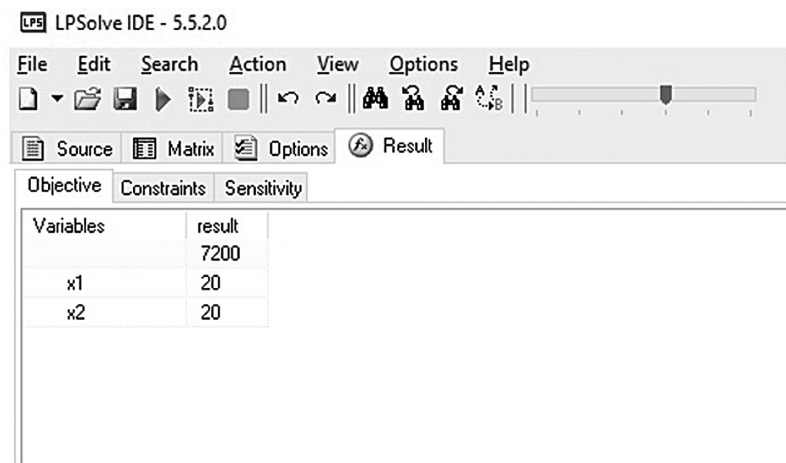


Figure 1.7 Solution obtained for the problem in LPSolve IDE

Example 1.2

The 2000 tons of cargo must be unloaded within 8 hours. Two cranes with capacities equal to 160 t/h and 200 t/h, respectively, can be used for unloading, with average cost per hour for the first crane of \$100/h, and for the second, \$150/h. Determine the optimal working times of the cranes to minimize the total cost of the cranes used.

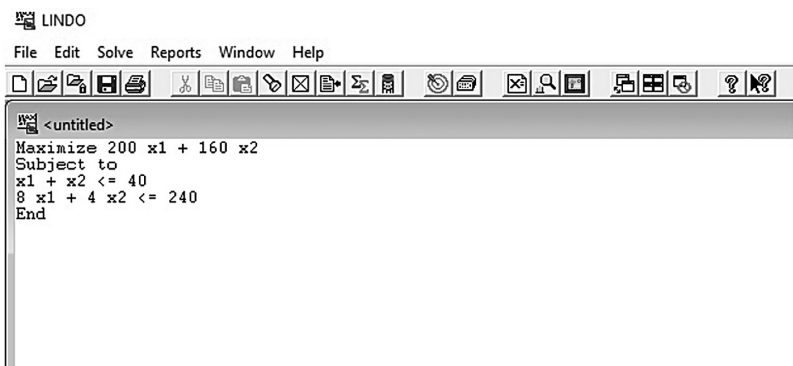


Figure 1.8 The linear program inserted in LINDO

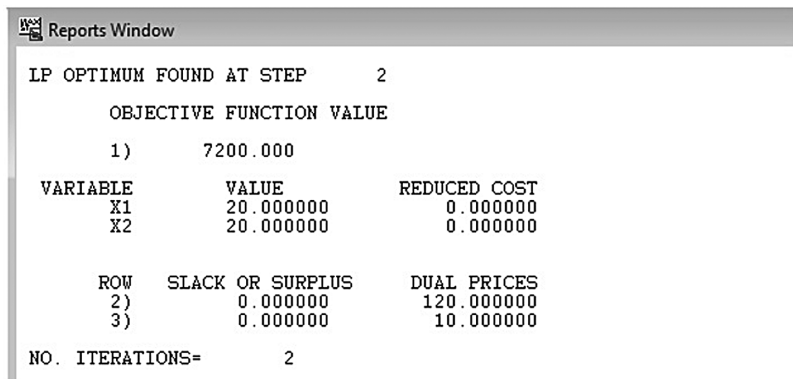


Figure 1.9 Solution of the problem obtained from LINDO

SOLUTION:

Let us denote variables x_1 and x_2 in the following way:

- x_1 – working time for the first crane (in hours)
- x_2 – working time for the second crane (in hours)

The linear program reads:

Minimize

$$F = 100 \cdot x_1 + 150 \cdot x_2$$

subject to:

$$160 \cdot x_1 + 200 \cdot x_2 \geq 2000$$

$$x_1 \leq 8$$

$$x_2 \leq 8$$

$$x_1 \geq 0, x_2 \geq 0$$

Since our problem has two decision variables, we present the problem graphically (Figure 1.10). By replacing “ $\leq$ ” and “ $\geq$ ” with “ $=$ ”, we create the equation for each constraint. We obtain the feasible solution space of the considered problem by taking into account all the constraints (Figure 1.10).

Figure 1.11 shows the solution space of the considered problem. To discover the optimal solution, we have to discover the feasible solution that has the smallest objective function value. In our case, the optimal solution is at the intersection of the following equations:

$$x_1 = 8 \text{ and } 160 \cdot x_1 + 200 \cdot x_2 = 2000$$

By solving the system of equations, we obtain the following: $x_1 = 8$ and $x_2 = 3.6$. The objective function value for this solution is equal to \$1340.

The linear program from the previous example can be written as the set of equations in the following way:

Minimize

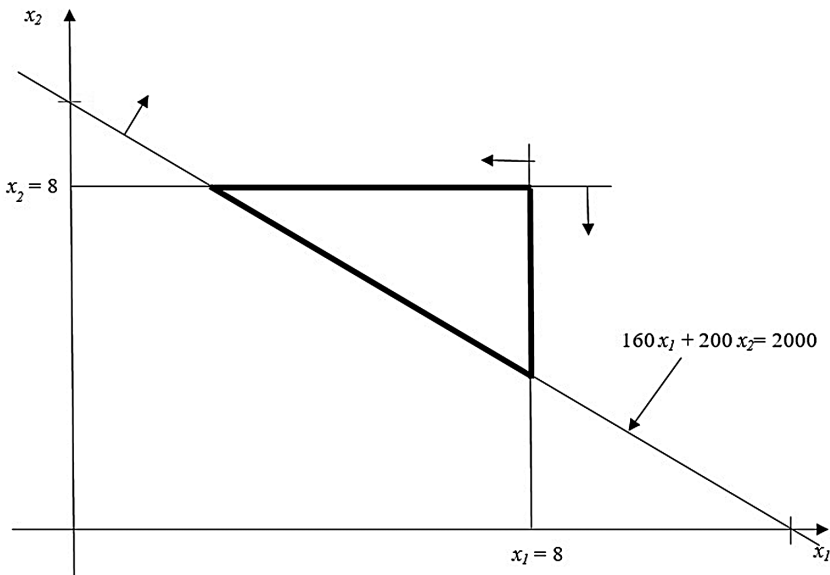


Figure 1.10 The feasible solutions space

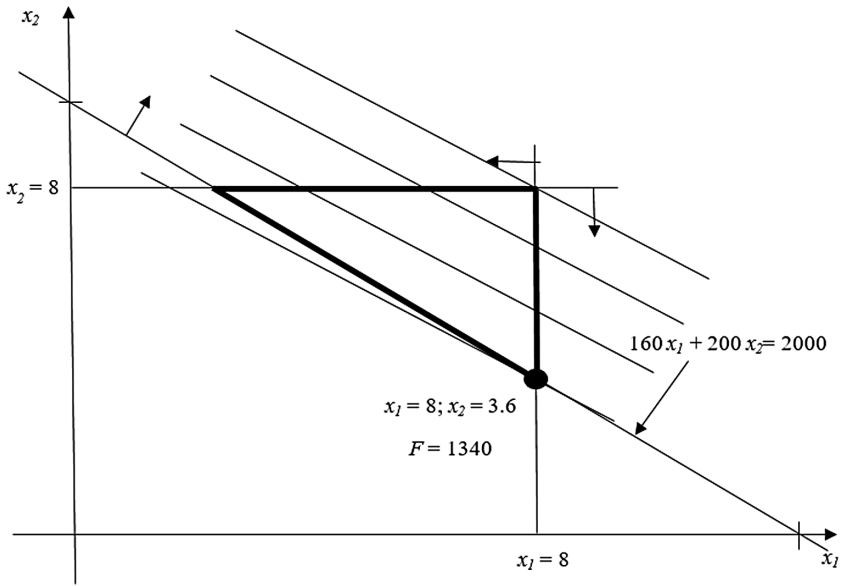


Figure 1.11 The solution of the problem

$$F = 100 \cdot x_1 + 150 \cdot x_2$$

subject to:

$$160 \cdot x_1 + 200 \cdot x_2 - x_3 = 2000$$

$$x_1 + x_4 = 8$$

$$x_2 + x_5 = 8$$

$$x_1 \geq 0, x_2 \geq 0, x_3 \geq 0, x_4 \geq 0, x_5 \geq 0$$

Variable x_3 is a surplus variable, and x_4 and x_5 are slack variables. Since we do not have a slack variable as the first constraint, we should add a new artificial variable in order to obtain the first feasible solution, i.e. to fill the simplex table. Let us denote this variable as R . The final value of R must be zero. Because of that, we add this variable to the objective function with a large positive coefficient. In general, we denote this coefficient as M . Now, our model has the following form:

Minimize

$$F = 100 \cdot x_1 + 150 \cdot x_2 + M \cdot R$$

subject to:

$$160 \cdot x_1 + 200 \cdot x_2 - x_3 + R = 2000$$

$$x_1 + x_4 = 8$$

$$x_2 + x_5 = 8$$

$$x_1 \geq 0, x_2 \geq 0, x_3 \geq 0, x_4 \geq 0, x_5 \geq 0$$

In this example we will take for M the value of 10000. Also, we will rewrite the objective function in a way that moves decision variables to the left-hand side of the sign “=”. Now, we have the following equations:

$$F - 100 \cdot x_1 - 150 \cdot x_2 - 10000 \cdot R = 0$$

$$160 \cdot x_1 + 200 \cdot x_2 - x_3 + R = 2000$$

$$x_1 + x_4 = 8$$

$$x_2 + x_5 = 8$$

We fill the simplex table by taking into account the equations obtained. In this table, we write the coefficients of the decision variables. Since we have four equations, the table has four rows (Table 1.5).

We note that the column R does not have an appropriate form (all elements must be equal to zero, except the element at the intersection with row R ; this element must be 1). Because of that, we have to make new values for row F in the following way: the elements in row R should be multiplied by the value for M (in our case, 10000). The value obtained should be added to the appropriate value of the element in row F . For example, the new elements in row F , are:

$$\text{for the column } F : 1 + 10000 \cdot 0 = 1$$

$$\text{for the column } x_1 : -100 + 10000 \cdot 160 = 1599900$$

$$\text{for the column } x_2 : -150 + 10000 \cdot 200 = 1999850$$

⋮

$$\text{for the column solution: } 0 + 10000 \cdot 200 = 2000000$$

Table 1.5 Initial simplex table

	F	x_1	x_2	x_3	x_4	x_5	R	<i>Solution</i>
F	1	-100	-150	0	0	0	-10000	0
R	0	160	200	-1	0	0	1	2000
x_4	0	1	0	0	1	0	0	8
x_5	0	0	1	0	0	1	0	8

Table 1.6 Simplex table with the first feasible solution

	<i>F</i>	x_1	x_2	x_3	x_4	x_5	<i>R</i>	<i>Solution</i>
<i>F</i>	1	1599900	1999850	-10000	0	0	0	20000000
<i>R</i>	0	160	200	-1	0	0	1	2000
x_4	0	1	0	0	1	0	0	8
x_5	0	0	1	0	0	1	0	8

The new simplex table has the form shown in Table 1.6.

We have to select the pivot column. For that purpose, we are going to select the decision variable that has the largest positive value in row *F*:

$$\max\{1599900, 1999850\} = 1999850 \Rightarrow \text{column } x_2$$

In the next step, we determine the pivot row. Rows with positive elements in the pivot column are potential pivot rows. Among the potential pivot rows, we will select the one that has the smallest value of the quotient right-hand sides, with the element in the pivot column:

$$\min\left\{\frac{2000}{200}, \frac{8}{1}\right\} = \min\{10, 8\} = 8 \Rightarrow \text{row } x_5$$

The new simplex table is given in Table 1.7. Since we still have positive values in row *F*, we should make a new iteration.

The pivot column will be x_1 since only this variable has a positive value in row *F*. The pivot row is obtained in the following way:

$$\min\left\{\frac{400}{160}, \frac{8}{1}\right\} = \min\{2.5, 8\} = 2.5 \Rightarrow \text{pivot row is } R$$

The pivot number is at the intersection of row *R* and column x_1 .

The new values for the elements of the table should be determined in the same way as in the previous step. The new simplex table is given in Table 1.8. We note again that the solution is not optimal as variable x_5 has positive values in row *F*. This time, the pivot element is at the intersection of the column x_5 and the row x_4 .

Table 1.7 Simplex table with the second feasible solution

	<i>F</i>	x_1	x_2	x_3	x_4	x_5	<i>R</i>	<i>Solution</i>
<i>F</i>	1	1599900	0	-10000	0	-1999850	0	4001200
<i>R</i>	0	160	0	-1	0	-200	1	400
x_4	0	1	0	0	1	0	0	8
x_2	0	0	1	0	0	1	0	8

The solution obtained in the new table (Table 1.9) is the optimal solution as the values of all decision variables in row F are negative or zero. From the table, we note the following:

$$\begin{aligned} F &= 1340, \\ x_1 &= 8, \\ x_2 &= 3.6, \text{ and} \\ x_4 &= 4.4. \end{aligned}$$

Let us show how this problem can be solved by Excel Solver, LPSolve IDE, and LINDO.

Figure 1.12 shows the appearance of the Excel sheet. Cells A2 and B2 will be used for the values of the decision variables x_1 and x_2 . Cell B4

Table 1.8 Simplex table with the third feasible solution

	F	x_1	x_2	x_3	x_4	x_5	R	<i>Solution</i>
F	1	0	0	-0.625	0	25	-9999.38	1450
x_1	0	1	0	-0.00625	0	-1.25	0.00625	2.5
x_4	0	0	0	0.00625	1	1.25	-0.00625	5.5
x_2	0	0	1	1	0	1	0	8

Table 1.9 Final simplex table

	F	x_1	x_2	x_3	x_4	x_5	R	<i>Solution</i>
F	1	0	0	-0.75	-20	0	-9999.25	1340
x_1	0	1	0	0	1	0	0	8
x_4	0	0	0	0.005	0.8	1	-0.005	4.4
x_2	0	0	1	0.995	-0.8	0	0.005	3.6

	A	B	C	D	E
1	x1	x2			
2					
3					
4	F =	0			
5					
6					
7					
8	0	>=	2000		
9	0	<=	8		
10	0	<=	8		
11					
12					
13					

Figure 1.12 The appearance of the Excel sheet

will be used for the objective function value. We use cells A8, A9, A10, C8, C9, and C10 for the constraints. Let us explain how cells B4, A8, A9, and A10 should be completed. In the cell, we write the function that calculates the value of the objective function $F = 100 \cdot x_1 + 150 \cdot x_2$. In our sheet, we use cells A2 and B2 for the x_1 and x_2 values, respectively. Our function for cell B4 is: “=100*A2+150*B2”. In a similar way, we can write the function for cell A8. In this cell, we calculate a part of the first constraint to the left of the sign “ $\geq$ ”, i.e., we calculate: $160 \cdot x_1 + 200 \cdot x_2$. This function can be written in the following way: “=160*A2+200*B2”. To the left of the sign “ $\leq$ ” in the second constraint is just x_2 . The function for the cell A9 is written in the following way: “= A2”. In a similar way, we write the function for the cell A10: “= B2”.

When the sheet is completed, we start the Solver (from the pallet data). In the window of the Solver, we should:

- enter the cell of the objective function,
- choose whether we maximize or minimize the objective function,
- enter constraints, and
- choose the solving method.

Figure 1.13 shows what Solver looks like after entering the values related to our example. After using the button Solve, we obtain the

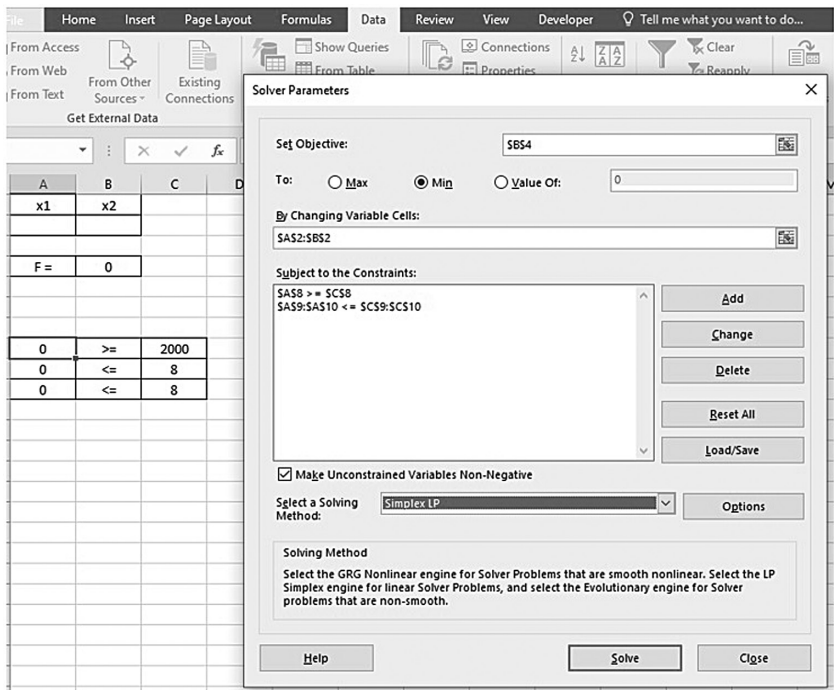


Figure 1.13 Excel Solver

	A	B	C	D	E
1	x1	x2			
2	8	3.6			
3					
4	F =	1340			
5					
6					
7					
8	2000	>=	2000		
9	8	<=	8		
10	3.6	<=	8		
11					
12					

Figure 1.14 The solution obtained

solution given in Figure 1.14. The solution is: $x_1=8$, $x_2=3.6$, and $F=1340$.

In the LPSolve IDE, we write the objective function and the constraints in a similar way as we write on paper. Figure 1.15 shows the implementation of our linear program. When insertion of the objective function and constraints is completed, we press the button Solve. The solution is given in Figure 1.16.

In a similar way, we can solve the linear program in LINDO. Figure 1.17 shows the implementation of our example. In the first row, we write the criteria for optimization and the objective function. After

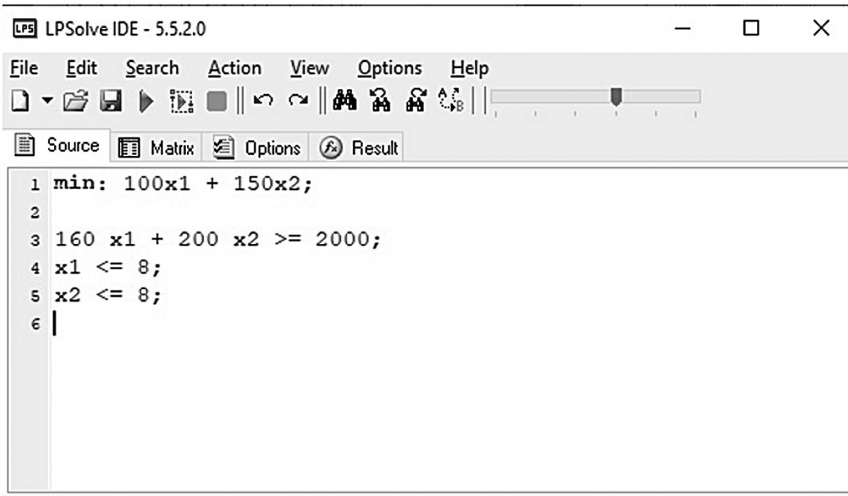


Figure 1.15 Linear program implementation in the LPSolve IDE

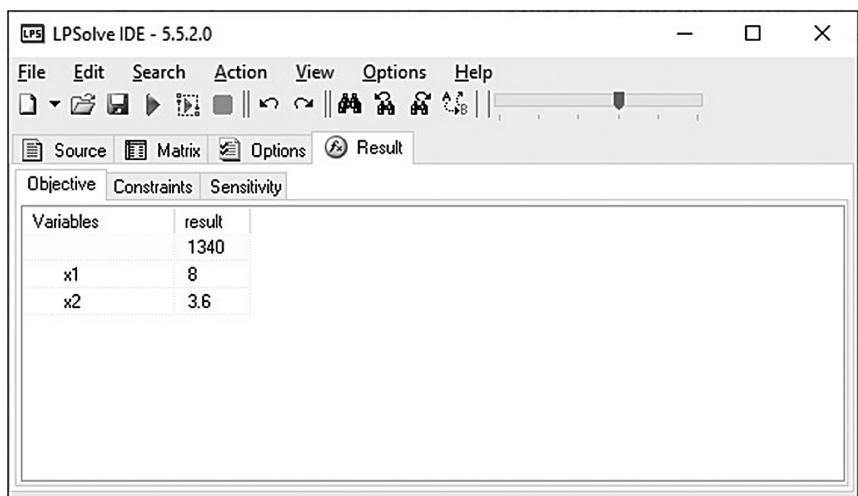


Figure 1.16 The solution obtained in the LPSolve IDE

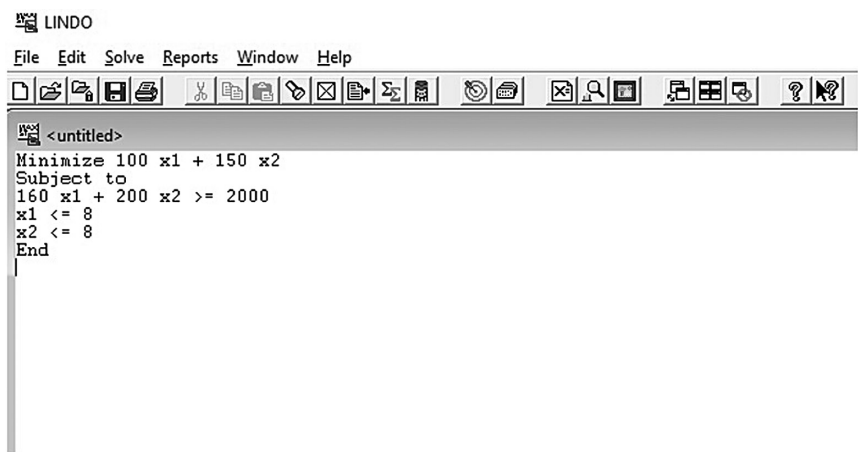


Figure 1.17 Linear program in Lindo

that, we write the set of constraints. Before we start with the constraints, we have to set “Subject to”, and, at the end of the set of constraints, we have to write “End”.

1.3 INTEGER AND MIXED-INTEGER PROGRAMMING

Analysts frequently realize that some or all of the variables in the formulated linear program must be *integers*. This means that some or all variables

take exclusively integer values. In order to make the formulated problem easier, analysts often allow these variables to take fractional values. For example, an analyst knows that the number of first-class passengers must be in the range between 30 and 40. A linear program could produce the “optimal solution” that tells us that the number of first-class passengers equals 37.8. In this case, we can neglect the fractional part, and we can decide to preserve 37 (or 38) seats for the first-class passengers. In this way, we are making a small numerical error, but we are capable of solving the problem easily. In some other situations, it is not possible for the analyst to behave in this way. Imagine that you have to decide about the alignment of a new highway. You must choose one out of numerous generated alternatives. This is a kind of “yes/no” (“1/0”) decisions: “Yes”, if the alternative is chosen, “No”, if otherwise. In other words, we can introduce binary variables into the analysis. The variable has a value of 1 if the i -th alternative is chosen, and a value of 0 otherwise, i.e.:

$$x_i = \begin{cases} 1 & \text{if the } i\text{-th alternative is chosen} \\ 0 & \text{otherwise} \end{cases}$$

The value 0.7, for example, of the variable means nothing to us. We are not able to decide about the best highway alignment if the variables take fractional values.

There are various logical constraints that should be taken into account when handling variables that take exclusively integer values. For example, in a situation where the decision maker has to choose, at most, one alternative among n available alternatives, the constraint reads:

$$\sum_{i=1}^n x_i \leq 1$$

In some situations, at least one alternative must be chosen among n alternatives. This constraint reads:

$$\sum_{i=1}^n x_i \geq 1$$

When we solve problems similar to the highway alignment problem, we work exclusively with integer variables. These kinds of problems are known as *integer programs*, and the corresponding area is known as *integer programming*. Integer programs usually describe the problems in which one, or more, alternatives must be selected from a finite set of generated alternatives. There are also problems in which some variables can take only integer values, whereas some other variables can take fractional values. These problems are known as *mixed-integer programs*. It is much harder to solve integer programming problems than linear programming problems.

The following is an integer programming model:

Maximize

$$F = \sum_{j=1}^n c_j x_j$$

subject to:

$$\sum_{j=1}^n a_{ij} x_j \leq b_i \quad \forall i = 1, 2, \dots, m$$

$$x_j \geq 0 \quad \text{and integer } \forall j = 1, 2, \dots, n$$

There are numerous software systems that solve linear, integer, and mixed-integer linear programs (CPLEX, Excel and Quattro Pro Solvers, FortMP, LAMPS, LINDO, LINGO, MILP88, MINTO, MIPIII, MPSIII, OML, and OSL).

1.3.1 Branch-and-bound method

The branch-and-bound method has been one of the most-used techniques for solving nondeterministic polynomial-time (NP)-hard optimization problems for years. The method was proposed by Ailsa Land and Alison Doig in 1960. The name of the method was suggested by Little et al. (1963). This method solves integer problems by considering their continuous version. In the first step, the problem identified is solved as a continuous model.

Let us note, for example, an integer-constrained variable x_i . Let us denote, by x_i^* , its optimum continuous value, which is fractional. Obviously, there are no feasible integer solutions within the following interval:

$$\lfloor x_i^* \rfloor < x_i < \lfloor x_i^* \rfloor + 1$$

Therefore, a feasible integer value of x_i must satisfy one of the following two conditions:

$$x_i \leq \lfloor x_i^* \rfloor$$

or

$$x_i \geq \lfloor x_i^* \rfloor + 1$$

When constraint $x_i \leq \lfloor x_i^* \rfloor$ and constraint $x_i \geq \lfloor x_i^* \rfloor + 1$ are imposed on the original solution space, two mutually exclusive problems are generated

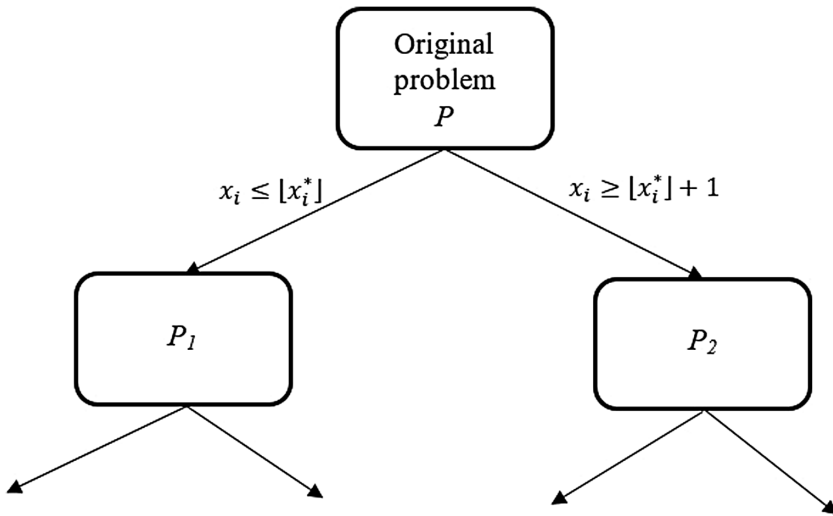


Figure 1.18 The original problem, P , is partitioned into two sub-problems P_1 and P_2

(Figure 1.18). In this way, an original problem P is partitioned into two sub-problems, P_1 and P_2 . The term “partitioned” is most often replaced by the term “branched”.

In the next step, we solve the sub-problems P_1 and P_2 . These sub-problems are solved as linear programs. If the optimal solution for the sub-problem obtained is feasible, with respect to the integer problem, it is stored and has the status of the best-known solution. In this case, there is no need to further branch this sub-problem. In the opposite case, the sub-problem is again branched into two sub-problems (Figure 1.18). In the same way, as we did for the variable x_i , we, once more, impose the integer conditions on one of the integer variables that, at present, has a fractional optimal value. Every time we discover a feasible integer solution better than the best-known solution, it becomes a new best-known solution.

The process of branching continues in the search for an optimal solution to the analyzed problem. This process is stopped in the following cases: (a) each sub-problem ends with an integer solution; (b) the analyst becomes aware that a better solution cannot be obtained.

When does an analyst become aware that a better solution cannot be obtained? The branch-and-bound technique uses the concept of bounding. Let us imagine that the continuous optimum solution of a sub-problem is worse than the best-known integer solution. In this case, there is no need to further investigate the sub-problem. The branch-and-bound technique uses the objective function value of the best-known integer solution as a bound to reject poorer sub-problems.

Example 1.3

A company has to transport 180 pallets to a customer. Pallets could be transported in 40-foot or 20-foot containers. The company has five 40-foot containers and five 20-foot containers. The transportation cost for one 40-foot container equals \$1000 and, for one 20-foot container equals \$600. Taking into consideration the dimensions and weights of each pallet and the dimensions and capacity of the containers, it is decided that one 40-foot container can be loaded with 50 pallets, and one 20-foot container can be loaded with 22 pallets. Determine the number of 40-foot containers and the number of 20-foot containers that should be transported to minimize the total transportation cost.

SOLUTION:

Let us denote by x_1 the number of 40-foot containers that should be transported, and by x_2 the number of 20-foot containers that should be transported to the customer. The mathematical formulation of the problem reads:

Minimize

$$F = 1000x_1 + 600x_2$$

subject to:

$$50x_1 + 22x_2 \geq 180$$

$$x_1 \leq 5$$

$$x_2 \leq 5$$

$$x_1 \geq 0, x_2 \geq 0 \text{ and integer}$$

We solve this program by the branch-and-bound method. In the first step, we solve the previous program with continuous variables (x_1 and x_2). Let us denote this program by P. The program P reads:

Minimize

$$F = 1000x_1 + 600x_2$$

subject to:

$$50x_1 + 22x_2 \geq 180$$

$$x_1 \leq 5$$

$$x_2 \leq 5$$

$$x_1 \geq 0, x_2 \geq 0$$

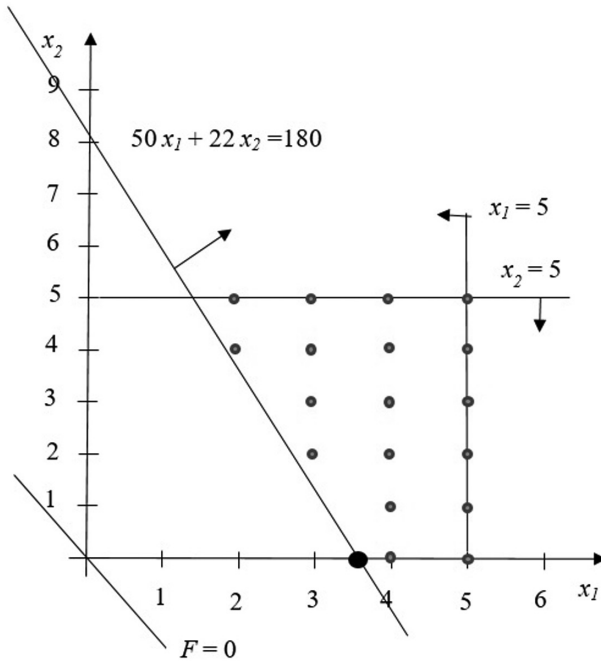


Figure 1.19 Graphical representation of the linear program P

The graphical representation of this problem is shown in Figure 1.19. The optimal solution of this program reads: $x_1 = 3.6$, $x_2 = 0$, and $F = 3600$. Since the variable x_1 is not an integer, we make two new programs. Let us denote them by P1 and P2. Program P1 consists of the objective function and the constraints of the program P and one new constraint, $x_1 \leq 3$. Program P2 consists of the objective function and the constraints of the program P and one new constraint, $x_1 \geq 4$.

Program P1 reads:
Minimize

$$F = 1000x_1 + 600x_2$$

subject to:

$$50x_1 + 22x_2 \geq 180$$

$$x_1 \leq 5$$

$$x_2 \leq 5$$

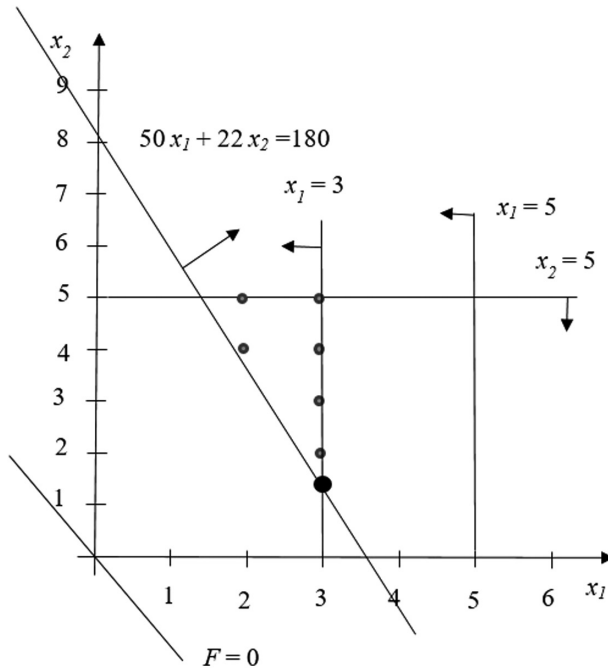


Figure 1.20 Graphical representation of the linear problem P1

$$x_1 \leq 3$$

$$x_1 \geq 0 \quad x_2 \geq 0$$

Figure 1.20 shows the graphical representation of the program P1. By solving this problem, we obtain the following solution: $x_1 = 3$, $x_2 = 1.364$, $F = 3818.18$. We can note that x_2 does not have an integer value. Consequently, we form two new programs, P11 and P12. Program P11 is generated by adding the constraint $x_2 \leq 1$ to the program P1. The program P12 is created by adding the constraint $x_2 \geq 2$ to the program P1.

The program P2 reads:
Minimize

$$F = 1000x_1 + 600x_2$$

subject to:

$$50x_1 + 22x_2 \geq 180$$

$$x_1 \leq 5$$

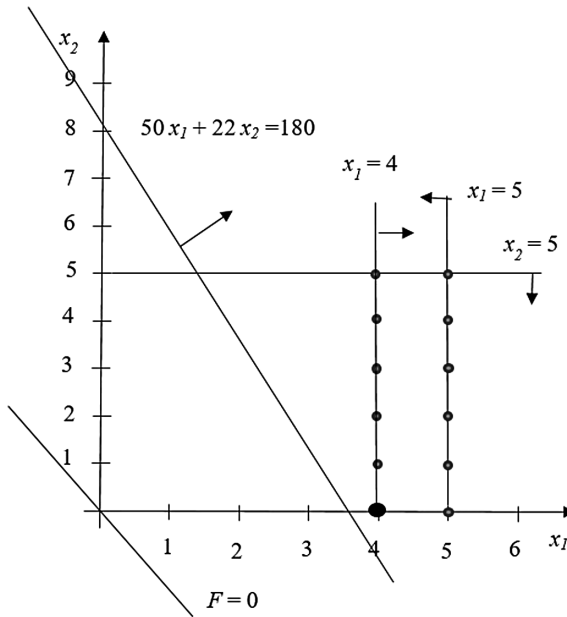


Figure 1.21 Graphical representation of the linear problem P2

$$x_2 \leq 5$$

$$x_1 \geq 4$$

$$x_1 \geq 0 \quad x_2 \geq 0$$

The graphical representation of this program is given in Figure 1.21. By solving this program, we obtain the following solution: $x_1=4$, $x_2=0$, and $F=4000$. The variables have integer values in this solution. Since we do not yet have a solution with integer values of the variables, this solution should be saved as the solution of the initial problem. Additionally, since both variables are integers, we do not have to make new programs.

Let us return to branch P1 and check the programs P11 and P12. The program P11 reads:

Minimize

$$F = 1000x_1 + 500x_2$$

subject to:

$$50x_1 + 22x_2 \geq 180$$

$$x_1 \leq 5$$

$$x_2 \leq 5$$

$$x_1 \leq 3$$

$$x_2 \leq 1$$

$$x_1 \geq 0 \quad x_2 \geq 0$$

The graphical representation of this program is given in Figure 1.22. We note that this program does not have a feasible solution.

Let us analyze the program P12. This program has the following form:

Minimize

$$F = 1000x_1 + 500x_2$$

subject to:

$$50x_1 + 22x_2 \geq 180$$

$$x_1 \leq 5$$

$$x_2 \leq 5$$

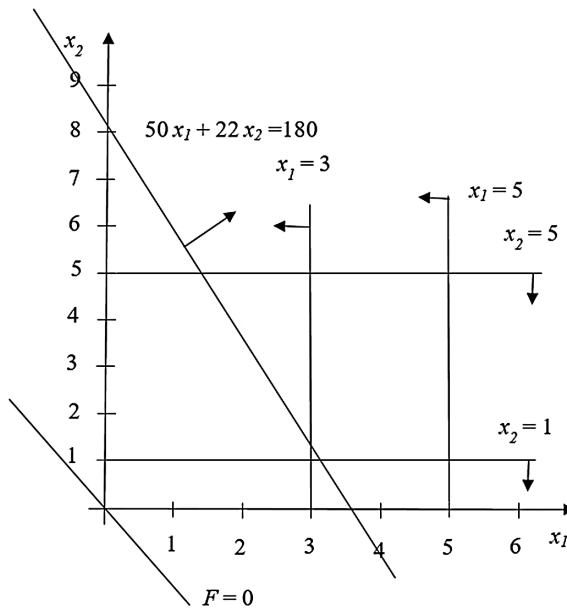


Figure 1.22 Graphical representation of the linear problem P11

$$x_1 \leq 3$$

$$x_2 \geq 2$$

$$x_1 \geq 0 \quad x_2 \geq 0$$

The graphical solution is given in Figure 1.23. The solution of this program is $x_1 = 2.72$, $x_2 = 2$, and $F = 3920$. The variable x_1 in this solution does not have an integer value and this program should be branched on two new programs, P121 and P122, by adding the new constraints $x_1 \leq 2$ and $x_1 \geq 3$ to the program P12.

The program P121 is:

Minimize

$$F = 1000x_1 + 500x_2$$

subject to:

$$50x_1 + 22x_2 \geq 180$$

$$x_1 \leq 5$$

$$x_2 \leq 5$$

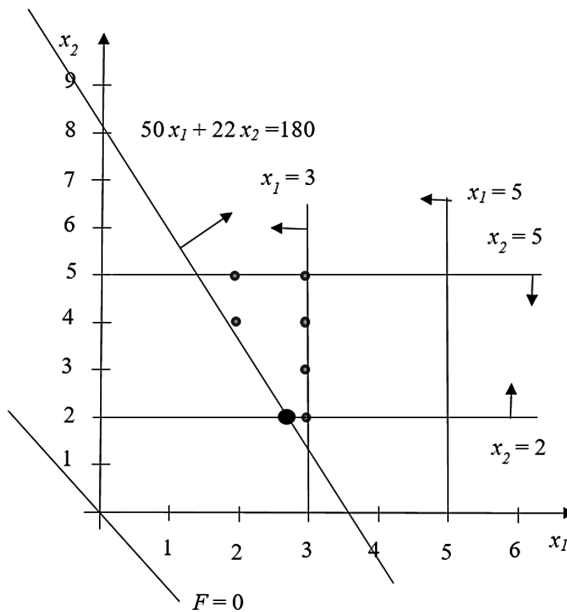


Figure 1.23 Graphical representation of the linear problem P12

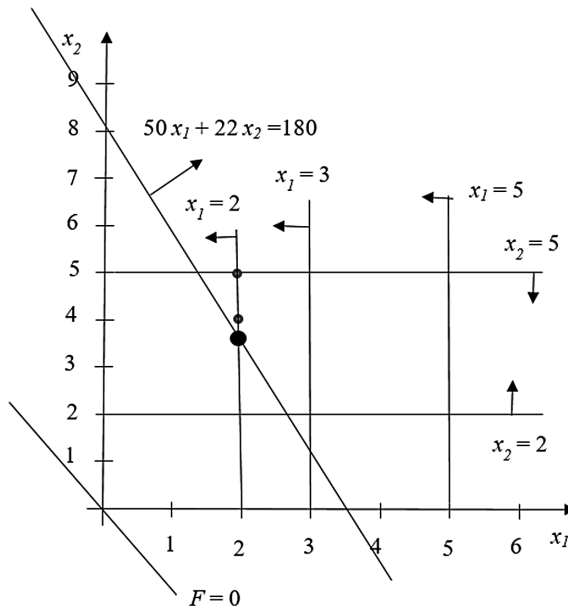


Figure 1.24 Graphical representation of the linear problem PI2I

$$x_1 \leq 3$$

$$x_2 \geq 2$$

$$x_1 \leq 2$$

$$x_1 \geq 0 \quad x_2 \geq 0$$

The graphical solution of this program is given in Figure 1.24. The solution of this program is $x_1=2$, $x_2=3.636$, and $F=4181.82$. We note that x_2 does not have an integer value. We make a new branch since the objective function of this program ($F=4181.82$) is greater than the objective function value of the previously found integer solution ($F=4000$), so that this branch will be bounded.

Solution reads:

$$x_1 = 2$$

$$x_2 = 3.636$$

$$F = 4181.82$$

As $4181.82 > 4000$, this branch should be bounded.

Let us check whether it is possible to find a better solution by solving program P122. This program reads:

Minimize

$$F = 1000x_1 + 500x_2$$

subject to:

$$50x_1 + 22x_2 \geq 180$$

$$x_1 \leq 5$$

$$x_2 \leq 5$$

$$x_1 \leq 3$$

$$x_2 \geq 2$$

$$x_1 \geq 3$$

$$x_1 \geq 0 \quad x_2 \geq 0$$

The graphical representation of this problem is given in Figure 1.25. The solution of this program is: $x_1=3$, $x_2=2$, and $F=4200$. We note

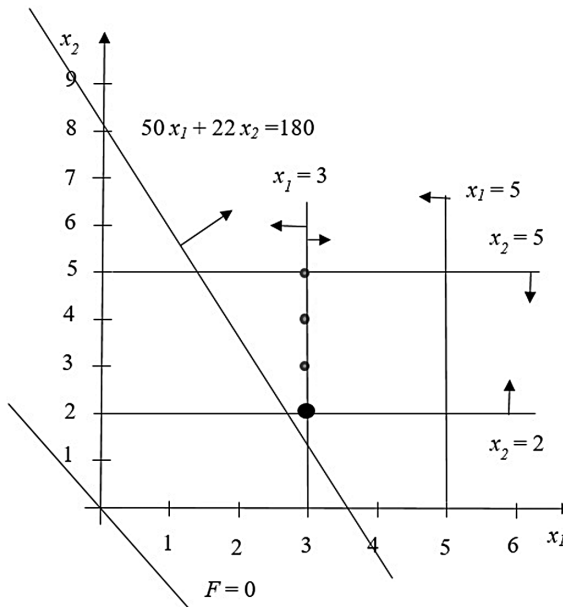


Figure 1.25 Graphical representation of the linear problem P122

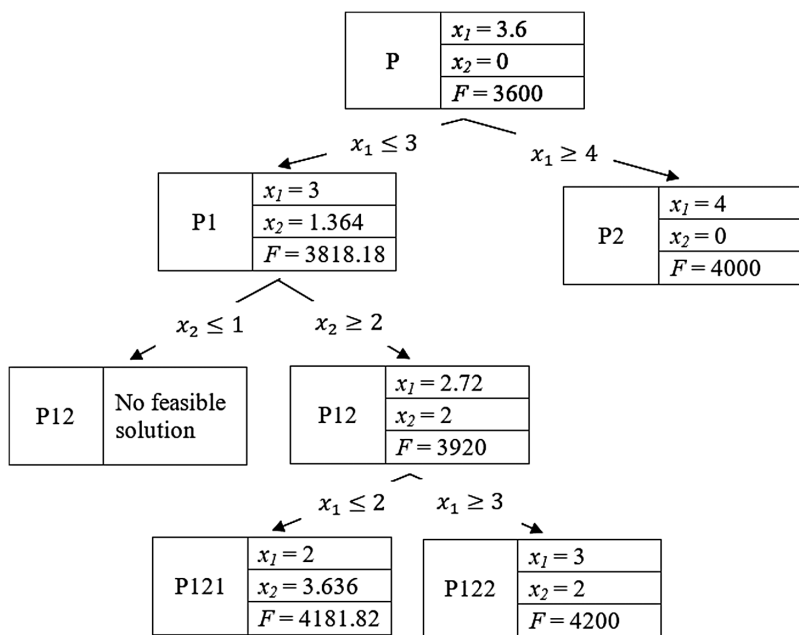


Figure 1.26 The overview of the branching procedure

that both variables have integer values. Since the objective function value is larger than the objective function value of the previously discovered integer solution ($F=4000$), we do not accept the new solution.

Since we do not have unsolved programs, we stop with the algorithm. We conclude that the solution of the initial problem is $x_1=4$, $x_2=0$, and $F=4000$. The overview of the branching procedure is shown in Figure 1.26.

The branch-and-bound method can be carried out using various software. Let us demonstrate how Excel Solver, LPSolver IDE, and LINDO can be used to solve the example in question.

Figure 1.27 shows the input data prepared for starting the Excel Solver. The functions on the cells B4, A7, A8, and A9 are as follows:

- cell B4: “=1000*A2 + 600*B2”
- cell A7: “=50*A2 + 22*B2”
- cell A8: “=A2”
- cell A9: “=B2”.

Figure 1.28 shows how input data should be inserted into Excel Solver. There is a small difference in the method of data insertion into the linear program and the integer program. For the integer programs, we have to add one more constraint in order to make integer variables.

	A	B	C	D	E
1	x1	x2			
2					
3					
4	F =	0			
5					
6					
7	0	>=	180		
8	0	<=	5		
9	0	<=	5		
10					
11					
12					

Figure 1.27 The prepared input data

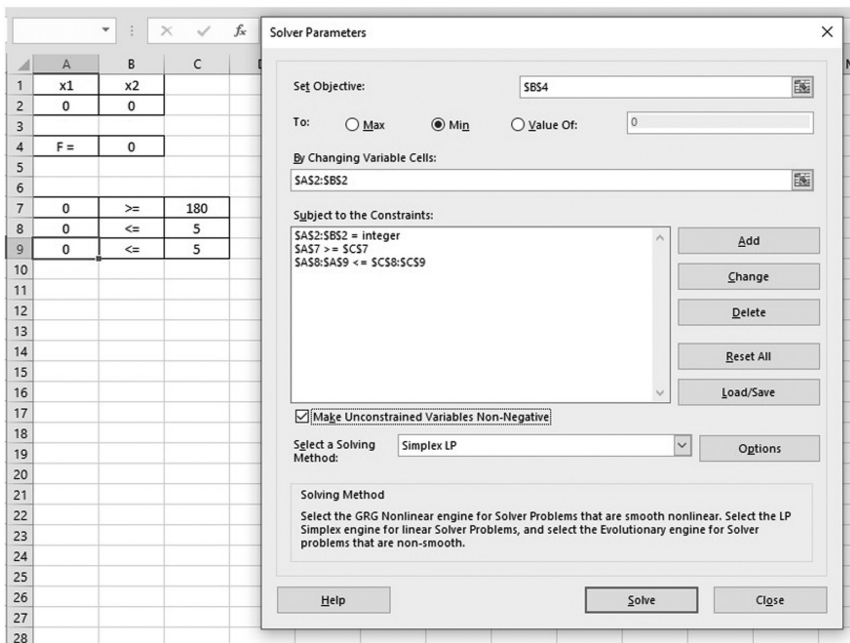


Figure 1.28 The Excel Solver for integer programming



Figure 1.29 Constraint that variables are integers

This step can be done in the way shown in Figure 1.29. The final solution of this example is given in Figure 1.30.

Figure 1.31 shows how the problem in question can be solved by LPSolve IDE software. The only difference between carrying out the linear and integer programs in LPSolve IDE is that one more constraint is added. In that constraint, all integer variables should be defined. In the example in question, the constraint is shown in line 7 of Figure 1.31.

The solution of this example in LPSolve IDE is given in Figure 1.32.

Let us demonstrate how the same problem can be solved in LINDO software. As we can see from Figure 1.33, the process is almost the same as for linear programming. Again, we must define variables as integers. This must be done as the end of the constraint. We define that a variable is an integer by writing “gin” in front of that variable (see Figure 1.33. We write “gin” in front of x_1 and x_2). In our example we define that both variables are integers. The solution obtained is given in Figure 1.34.

	A	B	C	D	E
1	x1	x2			
2	4	0			
3					
4	F =	4000			
5					
6					
7	200	>=	180		
8	4	<=	5		
9	0	<=	5		
10					
11					
12					

Figure 1.30 The solution obtained from the Excel Solver

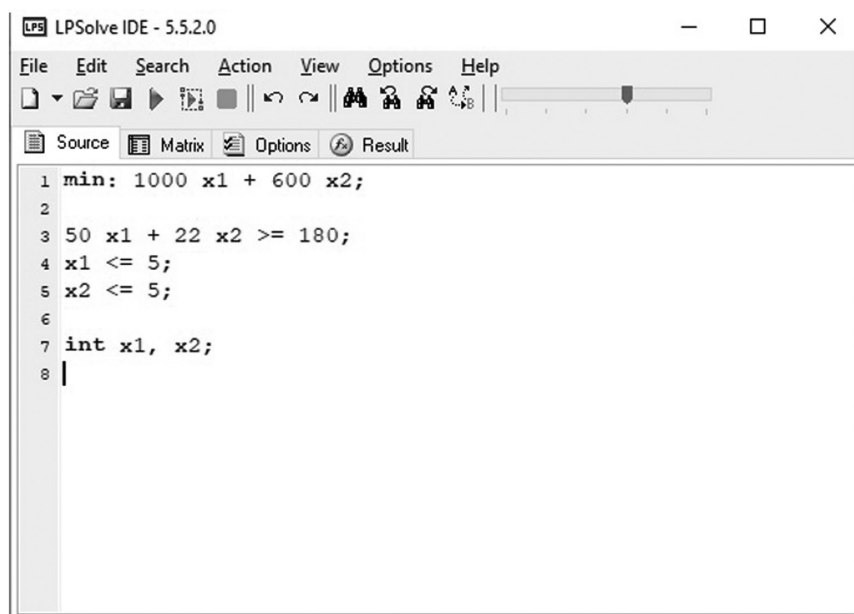


Figure 1.31 Integer program inserted in LPSolve IDE

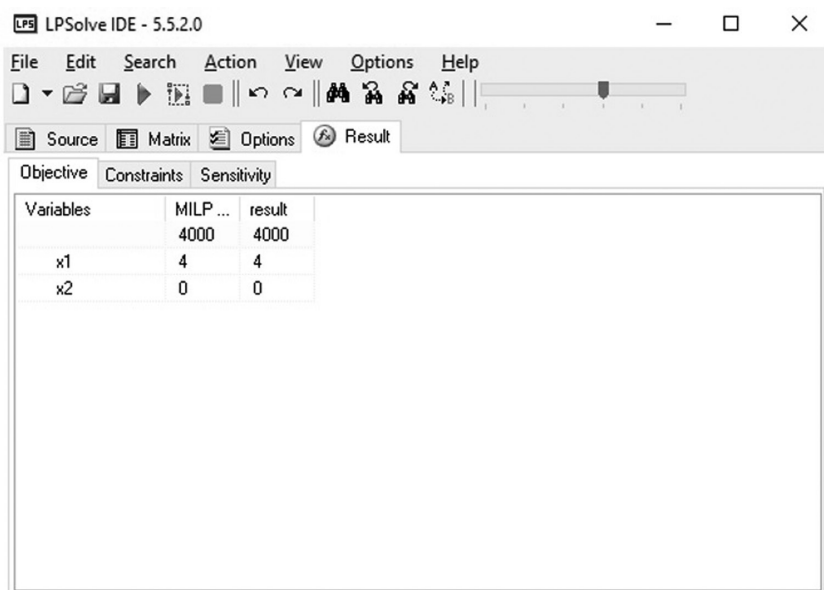


Figure 1.32 The solution of the problem obtained from LPSolve IDE

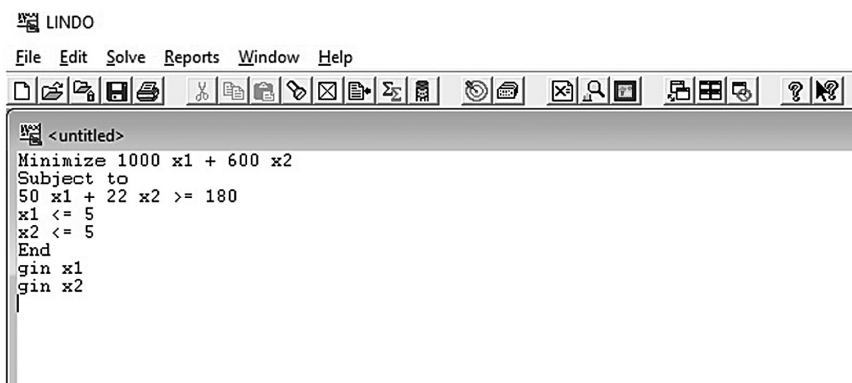


Figure 1.33 The Integer program inserted in LINDO software

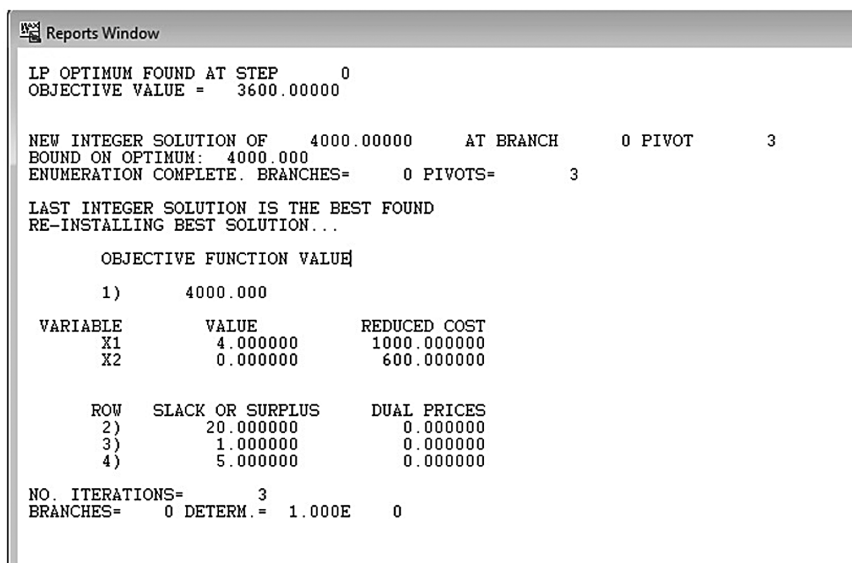


Figure 1.34 The solution obtained from LINDO software

Example 1.4

The liberalization of airline tariffs has led to intensive competition among air carriers. Under such conditions, an air carrier logically wants to sell the seats available in a way that maximizes profit. The liberalization of airline tariffs has also resulted in a large number of different tariffs existing on the same flight. Passengers paying lower tariffs (as a rule, making private trips) often reserve seats before passengers paying higher tariffs (business class passengers who decide to

travel several days or hours before the flight), which is why a certain number of passengers, who are prepared to pay a higher tariff, cannot find a vacant seat on the flight they want. The simplest reservation system is often called distinct fare class inventories, indicating separate seat inventories for each fare class. Once a seat is assigned to a fare class inventory, it may be booked only in that fare class, or else remain unsold. In the case of a nested reservation system, the high-fare request will not be rejected as long as any seats are available in lower fare classes. Let us consider the airline seat inventory control problem for a direct, nonstop flight. The capacity of an aircraft (the number of seats in the aircraft) equals 150. Let us assume that passengers are offered two tariff classes: \$130, and \$100. We assume that we are able to predict exactly the total number of requests for the two different passenger tariff classes. We expect 40 passenger requests for first class, and 120 passenger requests for second class. We decide to sell at least 15 seats to the passengers paying lower tariffs. We have to determine the total numbers of seats which must be sold in the different passenger tariff classes in order to achieve the maximum airline revenue.

SOLUTION:

Since we wish to determine the total numbers of seats sold in each of the different passenger tariff classes, the variables of the model can be defined as:

- x_1 – the total number of seats planned to be sold in the first-class passenger tariff
- x_2 – the total number of seats planned to be sold in the second-class passenger tariff

Since each seat from the first class tariff sells for \$130, the total revenue from selling x_1 seats is $130 \cdot x_1$. In the same way, the total airline revenue from x_2 seats in second class is equal to $100 \cdot x_2$. The total airline revenue equals the sum of the two revenues, i.e.: $130 \cdot x_1 + 100 \cdot x_2$.

From the problem formulation, we conclude that there are specific restrictions on seat sales and on demand. The seat sales restrictions may be expressed verbally in the following way:

- Total number of seats sold in both classes together must be less than or equal to the aircraft capacity.
- Total number of seats sold in any class must be less than or equal to the total number of passenger requests.
- Total number of seats sold in second class must be at least 15.
- Total number of seats sold in first class and in second class cannot be less than zero (non-negativity constraint).

The following is the mathematical model for airline revenue management problem:

Maximize

$$F = 130x_1 + 100x_2$$

subject to

$$x_1 + x_2 \leq 150$$

$$x_1 \leq 40$$

$$x_2 \leq 120$$

$$x_2 \geq 15$$

$$x_i \geq 0 \quad \text{and integer} \quad i = 1, 2$$

The implementation of this integer program in LPSolve IDE software is given in Figure 1.35. By solving this problem, we obtain the following solution: $x_1 = 40$, $x_2 = 110$. The objective function value equals \$16200.

Example 1.5

Many departing passengers walk significant distances in airport terminal buildings between check-in desks and gates (the word “gate” is used in the literature to describe aircraft stands at the airport terminals, as well as “off-pier stands” at the apron). Simultaneously, the distances walked by arriving passengers, between gates and the baggage claim area, can be also considerable. In addition, many transit passengers are expected to walk significant distances between specific

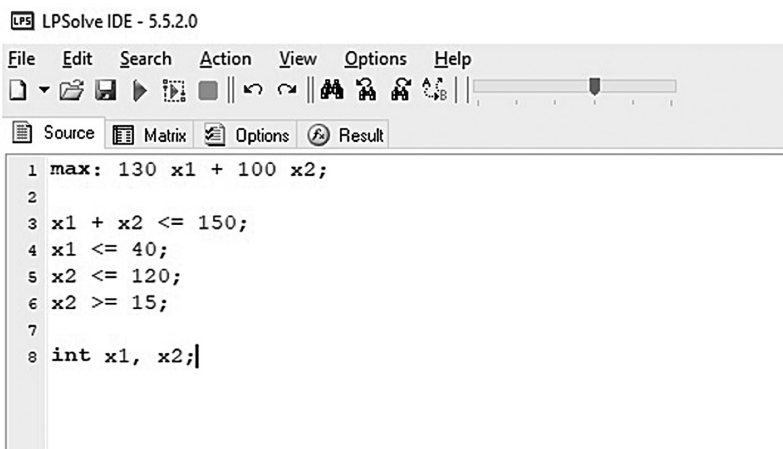


Figure 1.35 The implementation of the integer program in question in LPSolve IDE software

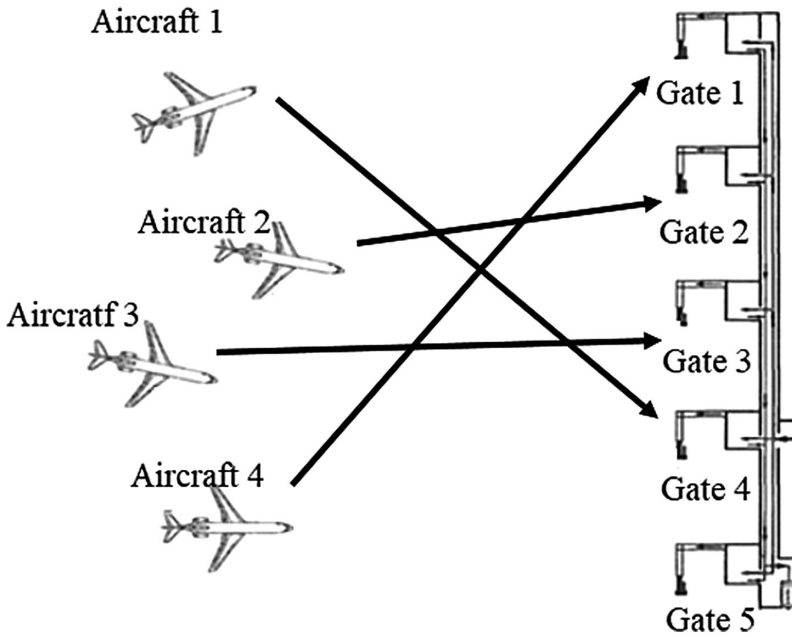


Figure 1.36 Graphical representation of the solution of the gate assignment problem

gates when changing planes (Figure 1.36). The total passenger walking distance within an airport terminal building may vary, depending on the specific assignment of aircraft to particular parking positions.

The standard *Gate Assignment Problem* can be defined in the following way. For a given set of parking positions and a given set of aircraft that can use any of these parking positions, find a parking position assignment for aircraft that will minimize the total walking distance of all passengers arriving, transiting and departing by aircraft parked at the set of parking positions. The decision maker must assign aircraft to available gates and must determine the start and end time of serving aircraft at the gate to which it has been assigned.

Airport gates are one of the greatest congestion points of the air transportation system. The total daily number of aircraft operations at large airports could be more than 1000, whereas the total number of gates is frequently more than 100.

Let us consider an example where we have to assign 4 aircraft to 5 available gates. Departure and arrival walking distances are given in Table 1.10.

The number of departing arriving passengers are given in Table 1.11.

SOLUTIONS:

For each pair of aircraft i and gates j we have to determine the total walking distance d_{ij} . According to data given in

Table 1.10 Walking distances for departure and arrival passengers

Gate	Walking distance for departure passengers between check-in desks and gate (m)	Walking distance for arrival passengers between gate and the baggage claim area (m)
1	320	250
2	405	360
3	290	300
4	150	320
5	500	275

Table 1.11 Numbers of departing and arriving passengers

Aircraft	Number of departing passengers	Number of arriving passengers
1	156	145
2	85	76
3	98	110
4	170	186

Table 1.12 Total walking distance (m) of all passengers for every aircraft-gate pair

d_{ij}	Gate 1	Gate 2	Gate 3	Gate 4	Gate 5
Aircraft 1	86170	115380	88740	69800	117875
Aircraft 2	46200	61785	47450	37070	63400
Aircraft 3	58860	79290	61420	49900	79250
Aircraft 4	100900	135810	105100	85020	136150

Table 1.10 and Table 1.11, these distances are calculated in the following way:

$$d_{11} = 156 \cdot 320 + 145 \cdot 250 = 86170$$

$$d_{12} = 156 \cdot 405 + 145 \cdot 360 = 115380$$

$$\vdots$$

$$d_{45} = 170 \cdot 500 + 186 \cdot 275 = 136150$$

Table 1.12 shows the total walking distance of all passengers for every aircraft-gate pair.

The Gate Assignment Problem is formulated in the following way:

Minimize

$$F = \sum_{i=1}^4 \sum_{j=1}^5 d_{ij} x_{ij}$$

subject to:

$$\sum_{j=1}^5 x_{ij} = 1 \quad \forall i = 1, 2, 3, 4$$

$$\sum_{i=1}^4 x_{ij} \leq 1 \quad \forall j = 1, 2, 3, 4, 5$$

$$x_{ij} \in \{0, 1\} \quad \forall i = 1, \dots, 4; j = 1, \dots, 5$$

The objective function represents the total walking distance. This objective function should be minimized. The first constraint guarantees that each aircraft will be served on one of the gates. A maximum of one aircraft can be served at each gate (as there are fewer aircraft than gates, this constraint has a sign $\leq$). This is defined by the second constraint. The third constraint defines the decision variables x_{ij} as being binary.

The mathematical formulation now reads:

Minimize

$$F = 86170x_{11} + 115380x_{12} + 88740x_{13} + \dots + 136150x_{45}$$

subject to:

$$x_{11} + x_{12} + x_{13} + x_{14} + x_{15} = 1$$

$$x_{21} + x_{22} + x_{23} + x_{24} + x_{25} = 1$$

$$x_{31} + x_{32} + x_{33} + x_{34} + x_{35} = 1$$

$$x_{41} + x_{42} + x_{43} + x_{44} + x_{45} = 1$$

$$x_{11} + x_{21} + x_{31} + x_{41} \leq 1$$

$$x_{12} + x_{22} + x_{32} + x_{42} \leq 1$$

$$x_{13} + x_{23} + x_{33} + x_{43} \leq 1$$

$$x_{14} + x_{24} + x_{34} + x_{44} \leq 1$$

$$x_{15} + x_{25} + x_{35} + x_{45} \leq 1$$

$$x_{ij} \in \{0, 1\} \quad \forall i = 1, \dots, 4; j = 1, \dots, 5$$

By solving this problem, we obtained the solution where x_{14} , x_{22} , x_{33} and x_{41} equal one, and all other variables are zero. This means that

Table 1.13 Characteristics of trips

Trip	Departure terminal	Departure time	Arrival terminal	Arrival time
1	<i>a</i>	7:00	<i>b</i>	8:00
2	<i>b</i>	7:30	<i>c</i>	8:15
3	<i>b</i>	8:05	<i>d</i>	8:50
4	<i>d</i>	9:00	<i>a</i>	10:00
5	<i>c</i>	9:10	<i>a</i>	9:55
6	<i>c</i>	9:20	<i>d</i>	10:05

gate 4 should be assigned to aircraft 1, gate 2 to aircraft 2, gate 3 to aircraft 3, and gate 1 to aircraft 4. The graphical representation of this solution is given in Figure 1.36. The total walking distance for this solution for all the passengers equals 293,905 m.

Example 1.6

(Modified from Ceder (2015))

There are two bus terminals *a* and *b*. There are six planned bus trips (Table 1.13 and Figure 1.37). The buses must be assigned to the set of planned timetabled trips in such a way as to minimize the total number

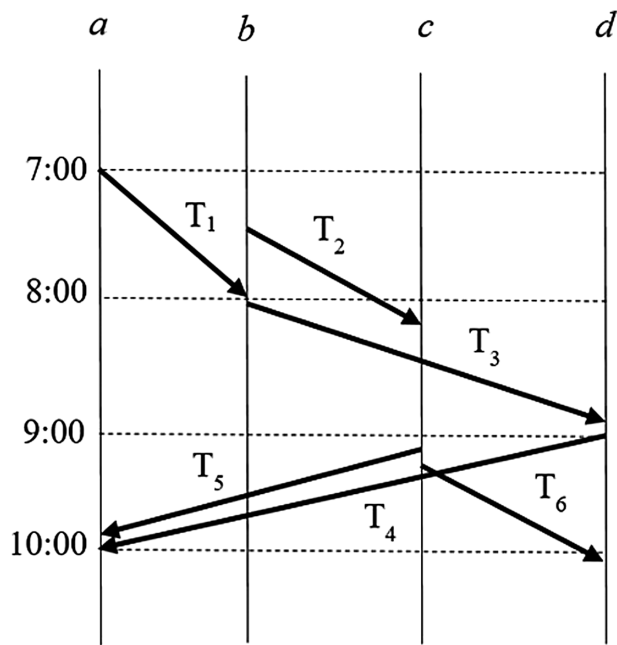


Figure 1.37 Graphical representation of the bus scheduling problem

Table 1.14 Travel times (min) of empty buses between the terminals

d_{ij}	a	b	c	d
a	0	50	40	50
b	50	0	35	40
c	40	35	0	35
d	50	40	35	0

of buses needed. The travel times of empty buses between terminals (d_{ij}) are given in Table 1.14.

SOLUTION:

Since buses have to perform 6 trips, we conclude that the number of buses necessary is six or less. We will need six buses if it is impossible to perform two trips by one bus. Our problem reads:

Maximize

$$F = \sum_{i \in T} \sum_{j \in T} x_{ij}$$

subject to:

$$\sum_{j \in T} x_{ij} \leq 1 \quad \forall i \in T$$

$$\sum_{i \in T} x_{ij} \leq 1 \quad \forall j \in T$$

$$x_{ij} \leq k_{ij} \quad \forall i, j \in T$$

$$x_{ij} \in \{0, 1\} \quad \forall i, j \in T$$

where:

T – set of trips

x_{ij} – decision variable that is equal to 1 if trips i and j should be joined, where the trip i is a predecessor of trip j , otherwise is equal to 0

k_{ij} – coefficient that takes values 1 or 0, according to the following expression:

$$k_{ij} = \begin{cases} 1, & \text{if } b_i + d_{ij} \leq a_j \\ 0, & \text{otherwise} \end{cases}.$$

The objective function represents the sum of the joined trips. The first constraint guarantees that each trip may be joined with a maximum of one successor trip. The second constraint defines that each trip may be joined with a maximum of one predecessor trip. If trips i and j cannot be joined, the decision variable x_{ij} is equal to 0 (third constraint). The fourth constraint defines decision variables x_{ij} as binary.

Figures 1.38 and 1.39 show how this problem can be solved in Excel. The following are the functions at the cells B22, from B17 to G17 and from H11 to H16:

For cell B22: “=SUM(B11:G16)”

For cell B17: “=SUM(B11:B16)”

...

For cell G17: “=SUM(G11:G16)”

For cell H11: “=SUM(B11:G11)”

...

For cell H16: “=SUM(B16:G16)”.

By solving the problem, we obtained the following solution: $x_{13} = 1$, $x_{34} = 1$, $x_{25} = 1$, with all other variables equaling zero. From this solution, we see that the first bus performs trips 1, 3, and 4. The second bus services trips 2 and 5. It is obvious that we need one more bus for trip 6. Consequently, the total number of buses is equal to three.

	A	B	C	D	E	F	G	H	I	J	K
1	kij	1	2	3	4	5	6				
2	1	0	0	1	1	1	1				
3	2	0	0	0	1	1	1				
4	3	0	0	0	1	0	0				
5	4	0	0	0	0	0	0				
6	5	0	0	0	0	0	0				
7	6	0	0	0	0	0	0				
8											
9											
10	xij	1	2	3	4	5	6				
11	1							0	<=	1	
12	2							0	<=	1	
13	3							0	<=	1	
14	4							0	<=	1	
15	5							0	<=	1	
16	6							0	<=	1	
17		0	0	0	0	0	0				
18		<=	<=	<=	<=	<=	<=				
19		1	1	1	1	1	1				
20											
21											
22	F =	0									
23											

Figure 1.38 The Excel spreadsheet

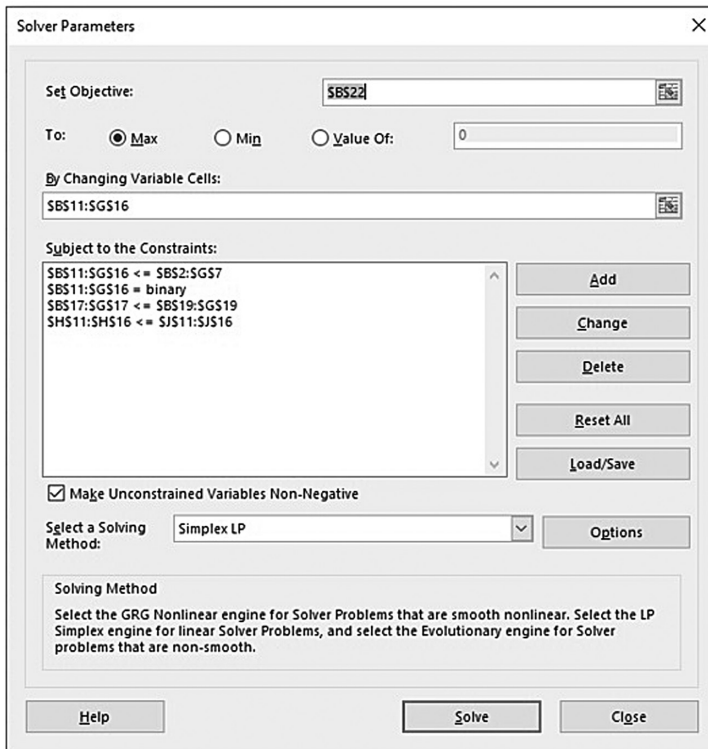


Figure 1.39 The completed Excel Solver

Example 1.7

There are three sections on a freeway. Capacities of the sections are given in Table 1.15. At each section, there is one on-ramp (Figure 1.40). The demands for on-ramps are given in Table 1.16. Before an on-ramp, the total traffic flow is $y = 4500$ vehicles/hour. Determine the maximal number of vehicles per hour that can enter the freeway from the on-ramps in the following two cases:

- from each on-ramp, at least half of the on-ramp demand vehicles must enter per hour.

Table 1.15 Capacity of each section of freeway

Section	Section capacity (vehicles/h)
1	5000
2	5700
3	6300

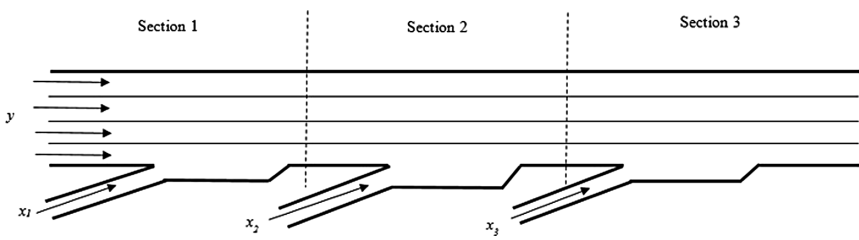


Figure 1.40 Three sections on a freeway

Table 1.16 Demand at on-ramps

Ramp	Demand (vehicles/h)
1	600
2	700
3	800

- b) from each on-ramp, the same percent of vehicles per hour must enter, taking into consideration on-ramp demand.

SOLUTION:

Let us denote by x_1 , x_2 , and x_3 the integer decision variables, which represent the number of vehicles per hour that can enter the freeway from the on-ramps 1, 2, and 3, respectively. The objective function can be written in the following way: $F = x_1 + x_2 + x_3$. This objective function, that represents the total number of vehicles per hour that enter from the on-ramps, should be maximized. Taking into account the capacities of the sections, we write the following constraints:

$$x_1 + y \leq 5000$$

$$x_1 + x_2 + y \leq 5700$$

$$x_1 + x_2 + x_3 + y \leq 6300$$

In the same way, we define three constraints that take care of on-ramp demands. Decision variables x_1 , x_2 , and x_3 must be smaller or equal to on-ramp demand, i.e.:

$$x_1 \leq 600$$

$$x_2 \leq 700$$

$$x_3 \leq 800$$

Let us define the constraints in the cases a and b.

- a) In this case, the variables x_1 , x_2 , and x_3 must be larger than or equal to half of the on-ramp demands, i.e.:

$$x_1 \geq 300$$

$$x_2 \geq 350$$

$$x_3 \geq 400$$

The integer program reads:

Maximize

$$F = x_1 + x_2 + x_3$$

subject to:

$$x_1 \leq 500$$

$$x_1 + x_2 \leq 1200$$

$$x_1 + x_2 + x_3 \leq 1800$$

$$x_1 \leq 600$$

$$x_2 \leq 700$$

$$x_3 \leq 800$$

$$x_1 \geq 300$$

$$x_2 \geq 350$$

$$x_3 \geq 400$$

$$x_i \geq 0 \text{ and integer } i = 1, 2, 3$$

The solution of this problem is $x_1=500$, $x_2=700$, $x_3=600$.

The objective function value equal 1800.

- b) In this case, there are requirements that, from each on-ramp, the same percent of vehicles must enter, taking into account on-ramp demands. Bearing this in mind, we write:

$$\frac{x_1}{600} = \frac{x_2}{700} = \frac{x_3}{800} = p$$

where p is the continuous decision variable. Multiplying this variable by 100, we can obtain the percent of on-ramp demands (in vehicles per hour) that can enter the freeway. From the previous relation, we write the following three constraints:

$$x_1 - 600p = 0$$

$$x_2 - 700p = 0$$

$$x_3 - 800p = 0$$

The whole model reads:
Maximize

$$F = x_1 + x_2 + x_3$$

subject to:

$$x_1 + y \leq 5000$$

$$x_1 + x_2 + y \leq 5700$$

$$x_1 + x_2 + x_3 + y \leq 6300$$

$$x_1 \leq 600$$

$$x_2 \leq 700$$

$$x_3 \leq 800$$

$$x_1 - 600p = 0$$

$$x_2 - 700p = 0$$

$$x_3 - 800p = 0$$

$$x_i \geq 0 \text{ and integer } i = 1, 2, 3$$

$$p \geq 0$$

The solution of this problem is $x_1=498$, $x_2=581$, $x_3=664$, $p=0.83$. The objective function value equals 1743.

We notice that in this example we had integer variables, and one continuous variable. Generally speaking, when some variables can be only integer values, and some others can be fractional values, such a problem is known as a *Mixed-Integer Program*. In transportation

engineering, mixed-integer programs are much more common than linear or integer programs. The next two examples that we will consider represent mixed-integer programs.

Example 1.8

There are 7 vehicles in the depot. All vehicles have the same capacity of 200 units. There are 25 nodes that need to be served. Characteristics of the nodes are given in Table 1.17. Every node should belong to just one route. Service at every node must begin within the specified time window. The depot is denoted as zero node. Determine a set of vehicles' routes that has the minimum total length.

Table 1.17 Node characteristics

Node number	Coordinate X	Coordinate Y	Capacity	Time window		Service time
0	40	50	0	0	1236	0
1	40	66	20	170	225	90
2	20	85	40	475	528	90
3	33	35	10	16	80	90
4	28	35	10	1001	1066	90
5	44	5	20	286	347	90
6	40	15	40	35	87	90
7	48	30	10	632	693	90
8	47	40	10	12	77	90
9	95	30	30	387	456	90
10	90	35	10	203	260	90
11	85	35	30	47	124	90
12	70	58	20	458	523	90
13	63	58	10	737	802	90
14	65	85	40	475	518	90
15	60	85	30	561	622	90
16	55	80	10	743	820	90
17	45	30	10	734	777	90
18	42	10	40	186	257	90
19	0	45	20	567	624	90
20	20	55	10	449	504	90
21	25	52	40	169	224	90
22	30	50	10	10	73	90
23	22	75	30	30	92	90
24	42	65	10	15	67	90
25	45	70	30	825	870	90

SOLUTION:

Our problem is known as the vehicle routing problem with time windows (VRPTW). The mathematical formulations of this problem is given in the following way (Desrochers et al., 1988; Cordeau et al., 2007):

Minimize

$$F = \sum_{k \in K} \sum_{(i,j) \in A} c_{ij} \cdot x_{ij}^k$$

subject to:

$$\sum_{k \in K} \sum_{j \in \delta^+(i)} x_{ij}^k = 1 \quad \forall i \in N$$

$$\sum_{j \in \delta^+(0)} x_{0j}^k = 1 \quad \forall k \in K$$

$$\sum_{i \in \delta^-(j)} x_{ij}^k - \sum_{i \in \delta^+(j)} x_{ji}^k = 0 \quad \forall k \in K, j \in N$$

$$\sum_{i \in \delta^-(n+1)} x_{i,n+1}^k = 1 \quad \forall k \in K$$

$$w_j^k \geq w_i^k + s_i + t_{ij} - M \cdot (1 - x_{ij}^k) \quad \forall k \in K, (i,j) \in A$$

$$a_i \leq w_i^k \leq b_i \quad \forall k \in K, i \in V$$

$$\sum_{i \in N} q_i \cdot \sum_{j \in \delta^+(i)} x_{ij}^k \leq Q \quad \forall k \in K$$

$$x_{ij}^k \in \{0,1\} \quad \forall k \in K, (i,j) \in A$$

where:

x_{ij}^k – binary variable that takes value 1 if vehicle k , after visiting node i , goes to node j

w_i^k – decision variable that denotes the beginning of the service of node i by vehicle k

Q – vehicle capacity

c_{ij} – transportation cost from node i to node j (or the length of the path from node i to node j)

a_i – earliest time point by which service can start at node i

b_i – latest time point by which service can start at node i

s_i – duration of the service at node i

q_i – demand of node i

K – set of vehicles

$$\delta^+(i) = \{j : (i, j) \in A\}$$

$$\delta^-(j) = \{i : (i, j) \in A\}$$

The objective function corresponds to the total transportation costs, that need to be minimized. The first constraint ensures that all customers will be served. The second and third constraints guarantee that all the vehicles depart and come back, respectively, to the depot. The third constraint is the conservation flow constraint. The fifth and sixth constraints take care of the time windows. The seventh constraint takes into account the available vehicle capacity. The eighth constraint defines decision variable x_{ij}^k as a binary variable.

By solving the mixed-integer program generated for the problem in question, we generated the seven routes, that have a total length of 549.07 distance units. The routes could be determined from the values of x variables. Variables x , that have values equal to 1, are given in Table 1.18. By taking into account these variables, we obtained the seven routes that are given in Table 1.19.

The routes obtained are shown in the Figure 1.41.

Example 1.9

In one day, 15 ships should arrive at the port. Their lengths (in m), arrival, and service times are given in Table 1.20. The length of the quay is 1000 m. Service of all ships must be finished within 1920 min

Table 1.18 Variables x which are equal to 1

Variable	Value	Variable	Value	Variable	Value
$x_{0,8}^1$	1	$x_{0,3}^4$	1	$x_{7,17}^6$	1
$x_{8,26}^1$	1	$x_{3,4}^4$	1	$x_{17,26}^6$	1
$x_{0,23}^2$	1	$x_{4,26}^4$	1	$x_{18,5}^6$	1
$x_{2,26}^2$	1	$x_{0,22}^5$	1	$x_{0,24}^7$	1
$x_{23,2}^2$	1	$x_{19,26}^5$	1	$x_{11,4}^7$	1
$x_{0,11}^3$	1	$x_{20,19}^5$	1	$x_{14,15}^7$	1
$x_{9,12}^3$	1	$x_{21,20}^5$	1	$x_{15,16}^7$	1
$x_{10,9}^3$	1	$x_{22,21}^5$	1	$x_{16,25}^7$	1
$x_{11,10}^3$	1	$x_{0,6}^6$	1	$x_{24,1}^7$	1
$x_{12,13}^3$	1	$x_{5,7}^6$	1	$x_{25,26}^7$	1
$x_{13,26}^3$	1	$x_{6,18}^6$	1		

Table 1.19 The optimal set of routes

Route 1	0 – 8 – 0
Route 2	0 – 23 – 2 – 0
Route 3	0 – 11 – 10 – 9 – 12 – 13 – 0
Route 4	0 – 3 – 4 – 0
Route 5	0 – 22 – 21 – 20 – 19 – 0
Route 6	0 – 6 – 18 – 5 – 7 – 17 – 0
Route 7	0 – 24 – 1 – 14 – 15 – 16 – 25 – 0

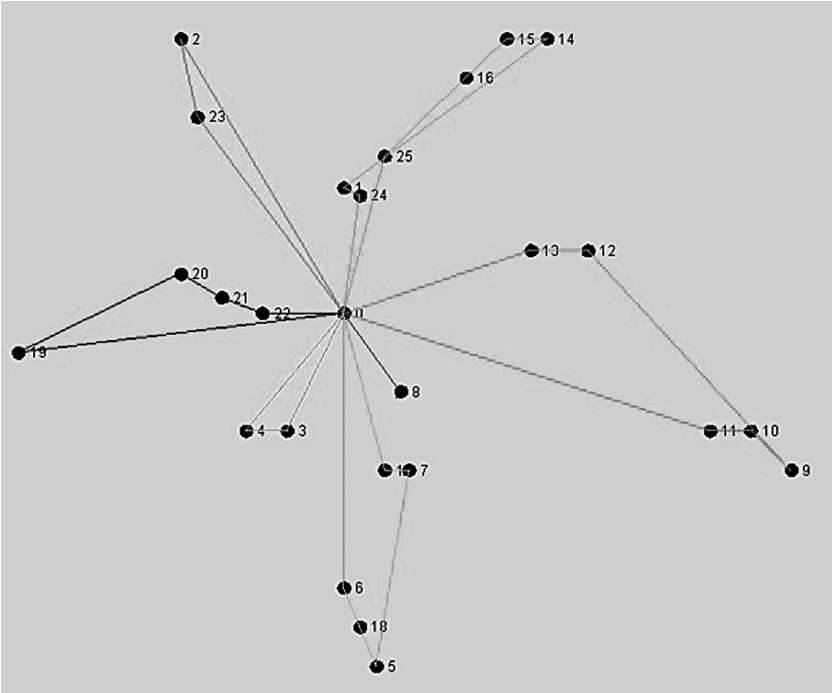


Figure 1.41 The optimal vehicle routes

of the beginning of the day. Determine the plan of ship services in a way that minimizes the total time that ships will spend at the port.

SOLUTION:

This problem is known in the literature as the berth allocation problem. When the length of the quay is given, instead of the fixed number of berths, the problem is known as the continuous berth allocation problem (as in the problem in question). In such a case, the ships' service plan is usually represented by a time–space diagram. One example of such a diagram is given in Figure 1.42.

Table 1.20 Characteristics of the ships

Ship	Length of ship (m)	Arrival time (min)	Service time (min)
1	150	15	300
2	200	60	240
3	250	90	360
4	100	210	180
5	300	255	360
6	150	360	480
7	200	410	180
8	250	480	540
9	130	590	300
10	180	750	240
11	200	1000	420
12	150	1215	360
13	180	1270	120
14	250	1370	540
15	200	1410	240

The mathematical formulation reads (Lee et al. 2010):

Minimize

$$F = \sum_{i=1}^N w_i \cdot (c_i - a_i)$$

subject to:

$$u_j - u_i - p_i - (\sigma_{ij} - 1) \cdot T \geq 0 \quad \forall 1 \leq i, j \leq N, i \neq j$$

$$v_j - v_i - s_i - (\delta_{ij} - 1) \cdot S \geq 0 \quad \forall 1 \leq i, j \leq N, i \neq j$$

$$\sigma_{ij} + \sigma_{ji} + \delta_{ij} + \delta_{ji} \geq 1 \quad \forall 1 \leq i, j \leq N, i \neq j$$

$$\sigma_{ij} + \sigma_{ji} \leq 1 \quad \forall 1 \leq i, j \leq N, i \neq j$$

$$\delta_{ij} + \delta_{ji} \leq 1 \quad \forall 1 \leq i, j \leq N, i \neq j$$

$$p_i + u_i = c_i \quad \forall 1 \leq i \leq N$$

$$a_i \leq u_i \leq (T - p_i) \quad u_i \in \mathfrak{R}^+, \forall 1 \leq i \leq N$$

$$0 \leq v_i \leq (S - s_i) \quad v_i \in \mathfrak{R}^+, \forall 1 \leq i \leq N$$

$$\sigma_{ij} \in \{0, 1\}, \delta_{ij} \in \{0, 1\} \quad \forall 1 \leq i, j \leq N, i \neq j$$

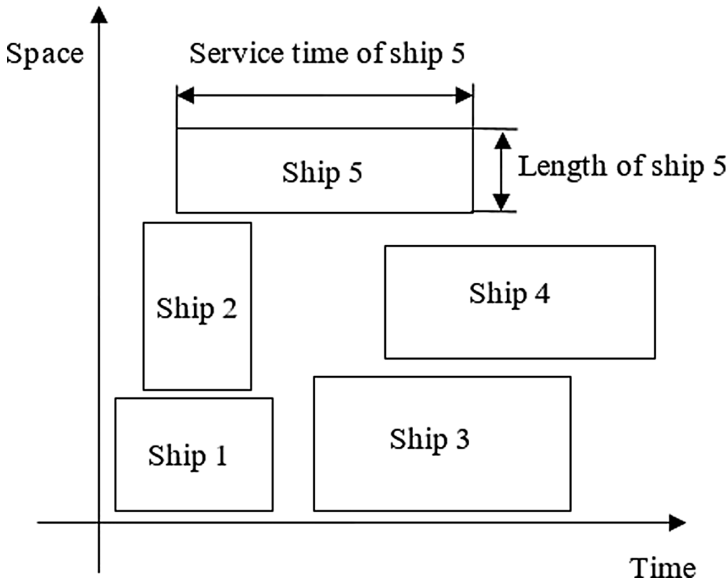


Figure 1.42 The example of the time–space diagram for continuous berth allocation problem

where:

- S – the length of the continuous berth
- T – the length of the planning horizon
- N – the total number of incoming ships,
- p_i – the service time for ship i , $1 \leq i \leq N$
- s_i – the length of ship i , $1 \leq i \leq N$
- a_i – arrival time of ship i , $1 \leq i \leq N$

and decision variables:

- w_i – the weight assigned for ship i , $1 \leq i \leq N$
- u_i – the mooring time of ship i , $1 \leq i \leq N$
- v_i – the starting berth position occupied by ship i , $1 \leq i \leq N$
- c_i – the departure time of ship i , $1 \leq i \leq N$

$$\sigma_{ij} = \begin{cases} 1, & \text{if ship } i \text{ is completely on the left of ship } j \text{ in the time-space diagram} \\ 0, & \text{otherwise} \end{cases}$$

$$\delta_{ij} = \begin{cases} 1, & \text{if ship } i \text{ is completely below ship } j \text{ in the time-space diagram} \\ 0, & \text{otherwise} \end{cases}$$

The objective function is the sum of the weighted turnaround times for all incoming ships. The first five constraints guarantee that the ships do not overlap in the time–space diagram. The completion time for

Table 1.21 The solution obtained for the continuous berth allocation problem

	v_i	u_i	c_i
Ship 1	0	15	315
Ship 2	250	60	300
Ship 3	750	90	450
Ship 4	150	210	390
Ship 5	450	255	615
Ship 6	300	360	840
Ship 7	100	410	590
Ship 8	750	480	1020
Ship 9	0	590	890
Ship 10	450	750	990
Ship 11	0	1000	1420
Ship 12	200	1215	1575
Ship 13	350	1270	1390
Ship 14	530	1370	1910
Ship 15	780	1410	1650

each ship (c_i) is calculated according to the sixth constraint. The feasible domain of the decision variables is defined by the seventh, eighth, and ninth constraints.

In our example, we do not have values for the weights of ships (the weight represents the importance that the decision maker gives to the specific ship). We assume that the weight of each ship is equal to 1. By solving the mixed-integer program based on previous mathematical formulations, we obtain the solution given in the Table 1.21. The table contains the values of the v_i , u_i and c_i decision variables. From these values, we can conclude where the ships should be located on the quay, when the service of each ship starts, and when the service of each ship is finished. The graphical representation of this solution is given in Figure 1.43.

1.4 DYNAMIC PROGRAMMING

In traffic engineering, we often encounter problems for which successful solutions need to be made by a series of decisions over time. For example, city administrations face capital budgeting problems. The city administration often needs to allocate the available budget to several different transport infrastructure projects. At the end of each year, a decision is made for the next year. Such and similar problems are most often solved in stages, where the stages are connected through recursive computation. Dynamic programming solves problem in stages. In this way, the whole problem is broken down into smaller problems.



Figure 1.43 The graphical representation of the solution of the continuous berth allocation problem

Let us clarify the basic principles of dynamic programming by analyzing the following example. International freight is usually shipped in inter-modal containers *via* multiple modes of transportation (rail, ship, truck). There is no handling of the freight itself when freight changes its mode of transportation. We consider shipping of one 20-foot container that should be shipped from the supplier (node 0) to the client (node 9). The company could send the container by truck to the railway station at node 1, to the railway station at node 2, or to the railway station at node 3 (Figure 1.44). From any of these railway stations, the container could be sent by rail to ports (node 4, node 5, or node 6). From any of these ports, the container could be sent by container ship to port 7 or port 8. Finally, from port 7 or port 8, the container could be sent by truck to the final destination (node 9).

The transportation costs along specific links are given in Table 1.22.

Let us consider, for example, container route (0, 2, 4, 8, 9). The total transportation costs of this container route are equal to $500 + 400 + 1,600 + 800 = 3,300$. The total transportation costs in the case of the container route (0, 3, 5, 7, 9) are equal to $600 + 350 + 1550 + 700 = 3,200$. Obviously, route (0, 2, 4, 8, 9) is cheaper than route (0, 3, 5, 7, 9). By proceeding in this way, we can examine all possible routes in the network, calculate their costs, and finally discover the cheapest route. We could perform exhaustive enumerations, i.e., we could compute the total cost for each of the possible container routes. The optimal solution would be the container route that has the lowest transportation costs. In the case of medium- and

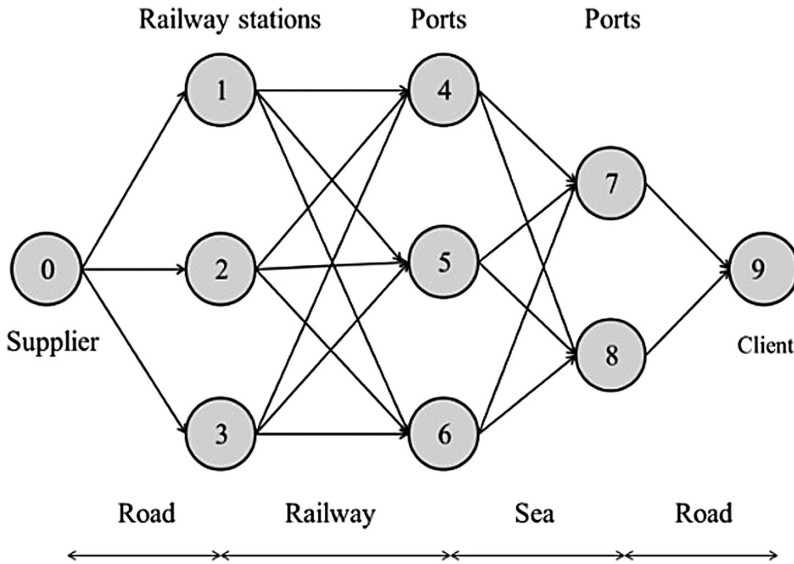


Figure 1.44 The transportation costs along specific links given in Table 1.22

Table 1.22 Transportation costs

Link	Cost (\$)	Link	Cost (\$)
(0, 1)	450	(3, 5)	350
(0, 2)	500	(3, 6)	300
(0, 3)	600	(4, 7)	1,500
(1, 4)	250	(4, 8)	1,600
(1, 5)	300	(5, 7)	1,550
(1, 6)	350	(5, 8)	1,700
(2, 4)	400	(6, 7)	1,450
(2, 5)	250	(6, 8)	1,850
(2, 6)	300	(7, 9)	700
(3, 4)	550	(8, 9)	800

large-sized transportation networks, exhaustive enumeration would lead to computational inefficiency. In addition, exhaustive enumeration does not use any information on the routes previously examined to remove future, poorer routes.

Our task is to discover the cheapest shipping route for the container. The cheapest shipping route problem could be solved in stages. Dynamic programming is a mathematical procedure that solves problems in stages (Figure 1.45). Dynamic programming was introduced in 1953 by Richard Ernest Bellman (1920–1984), an American applied mathematician. In each

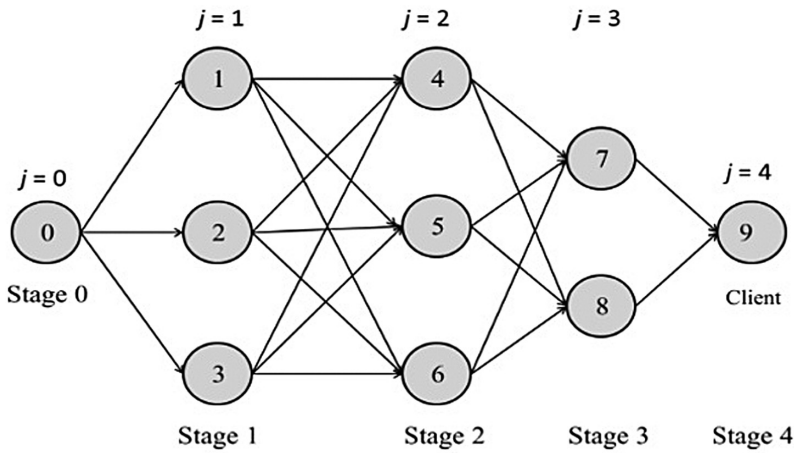


Figure 1.45 Network model of the cheapest shipping route problem

stage, the analyst considers one optimizing variable. The computations at the various stages are linked in such a way that the optimal solution of the problem in question is obtained when the final stage is reached.

Each point, where the container is being transferred from one transportation mode to the other, represents a stage. The initial stage ($j = 0$) represents the supplier. The first stage ($j = 1$) refers to the container transfer from the truck to the train. It can happen at node 1, node 2, or node 3. The second stage ($j = 2$) refers to the container transfer from the train to the container ship. This operation could happen at node 4, node 5, or node 6. The third stage ($j = 3$) refers to the container transfer from the container ship to the truck. This could occur at node 7 or node 8. Finally, the last stage ($j = 4$) represents the client.

Let us introduce the following notation:

x_1 – the total cost in stage 1

x_2 – the total cost in stage 1 and stage 2

x_3 – the total cost in stage 1, stage 2, and stage 3

x_4 – the total cost in stage 1, stage 2, stage 3, and stage 4

The lengths of the links connecting the nodes of consecutive stages represent the transportation cost of the link. The lengths of the links that connect stage 0 with the nodes in stage 1 are indicated in Figure 1.46.

Let us introduce the following notation:

$f_j(x_j)$ – the length of the shortest path that leads to node x_j at stage j .

The following is satisfied in the initial stage (supplier): $f_0(0) = 0$. We are now ready to calculate transportation costs stage by stage.

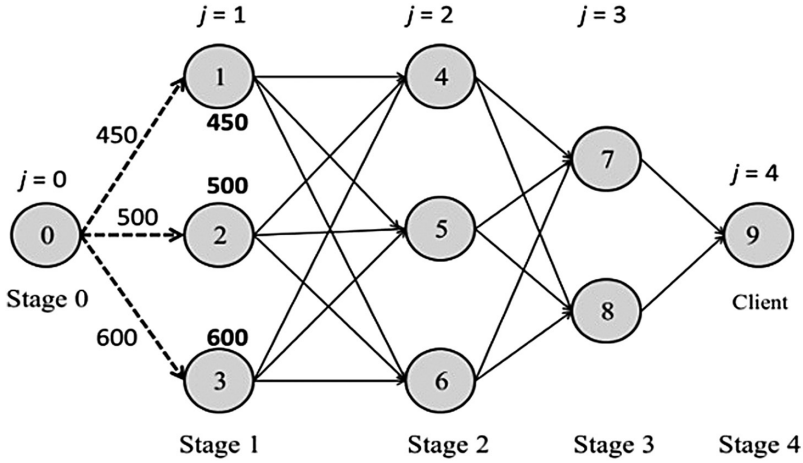


Figure 1.46 Calculating transportation costs of the first stage

Stage 1 (container transfer from the truck to the train):

The length of the shortest path that leads to node x_1 at stage 1 is:

$$f_1(x_1) = f_0(x_0) + C_1(0, x_1)$$

where $C_1(0, x_1)$ are the transportation costs of link $(0, x_1)$. We have the following:

$$f_1(1) = 0 + 450 = 450$$

$$f_1(2) = 0 + 500 = 500$$

$$f_1(3) = 0 + 600 = 600$$

The lengths of the shortest paths that lead to nodes 1, 2, and 3 are indicated in Figure 1.46 in bold letters.

Stage 2 (container transfer from the train to the ship):

We now calculate the lengths of the shortest paths to nodes 4, 5, and 6. All these nodes have three incoming links from the stage 1. Consequently, we write the following:

$$\left(\begin{array}{c} \text{length of shortest} \\ \text{path to node } x_2 \end{array} \right) = \min_{\substack{\text{feasible} \\ \text{links} \\ (x_1, x_2)}} \left\{ \left(\begin{array}{c} \text{length of shortest} \\ \text{path to node } x_1 \end{array} \right) + \left(\begin{array}{c} \text{length of link} \\ (x_1, x_2) \end{array} \right) \right\}$$

In other words:

$$f_2(x_2) = \min_{\substack{\text{feasible} \\ \text{links} \\ (x_1, x_2)}} \{f_1(x_1) + C_2(x_1, x_2)\}$$

i.e.:

$$\begin{aligned} f_2(4) &= \min(450 + 250, 500 + 400, 600 + 550) \\ &= \min(700, 900, 1150) = 700 \end{aligned}$$

$$\begin{aligned} f_2(5) &= \min(450 + 300, 500 + 250, 600 + 350) \\ &= \min(750, 750, 950) = 750 \end{aligned}$$

$$\begin{aligned} f_2(6) &= \min(450 + 350, 500 + 300, 600 + 300) \\ &= \min(800, 800, 900) = 800 \end{aligned}$$

Figure 1.47 illustrates the calculation of the transportation costs of the second stage.

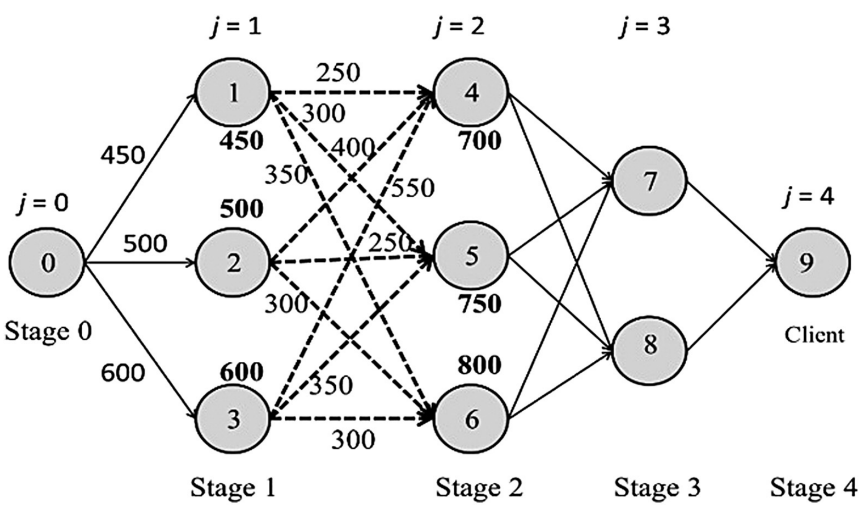


Figure 1.47 Calculating transportation costs of the second stage

Stage 3 (container transfer from the container ship to the truck):

The length of the shortest path $f_3(x_3)$ to node x_3 equals:

$$f_3(x_3) = \min_{\substack{\text{feasible} \\ \text{links} \\ (x_2, x_3)}} \{f_2(x_2) + C_3(x_2, x_3)\}$$

$$\begin{aligned} f_3(7) &= \min(700 + 1,500, 750 + 1,550, 800 + 1,450) \\ &= \min(2,200, 2,300, 2,250) = 2,200 \end{aligned}$$

$$\begin{aligned} f_3(8) &= \min(700 + 1,600, 750 + 1,700, 800 + 1,850) \\ &= \min(2,300, 2,450, 2,650) = 2,300 \end{aligned}$$

Figure 1.48 illustrates calculation of the transportation costs of the third stage.

Stage 4 (delivering container to the client):

The length of the shortest path $f_3(x_3)$ to node x_3 equals:

$$f_4(x_4) = \min_{\substack{\text{feasible} \\ \text{links} \\ (x_3, x_4)}} \{f_3(x_3) + C_4(x_3, x_4)\}$$

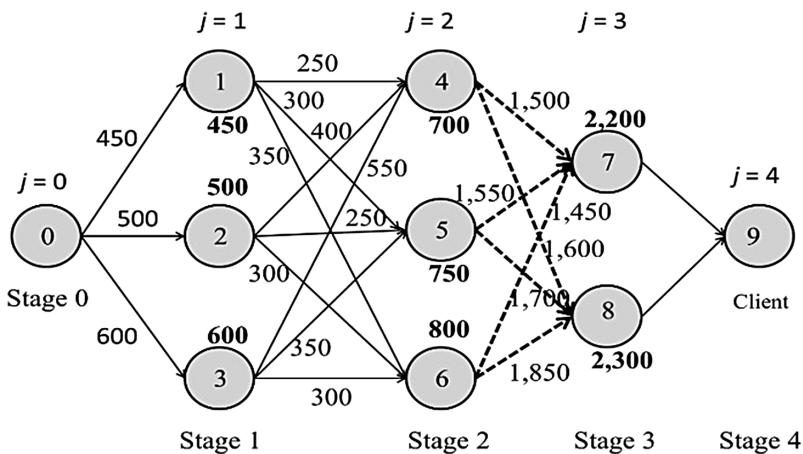


Figure 1.48 Calculating transportation costs of the third stage

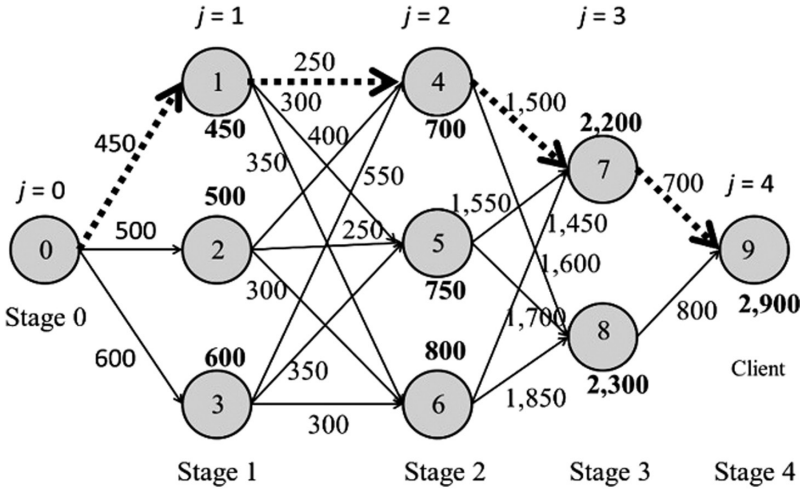


Figure 1.49 The cheapest shipping route for the container

i.e.:

$$f_4(9) = \min(2,200 + 700, 2,300 + 800) = \min(2,900, 3,100) = 2,900$$

The length of the shortest (cheapest) path from the supplier to the client is:

$$(0, 1, 4, 7, 9)$$

The optimal path is shown in Figure 1.49.

1.4.1 Forward and backward recursive equation

If we analyze the equations that we used to solve the container shipment problem, we see that they are recursive, since knowing $f_0(x_0)$, we compute $f_j(x_j)$ for stage j from $f_{j-1}(x_{j-1})$ at stage $j-1$. The dynamic programming recursive equations read:

$$f_0(x_0) = 0$$

$$f_j(x_j) = \min_{\text{feasible proposals } p_j} \{f_{j-1}(x_{j-1}) + C_j(p_j)\} \quad j = 1, \dots, n$$

The computations are carried out from the first stage to the last stage. This type of computation is called a forward procedure. Graphical representation of the forward pass is given in Figure 1.50.

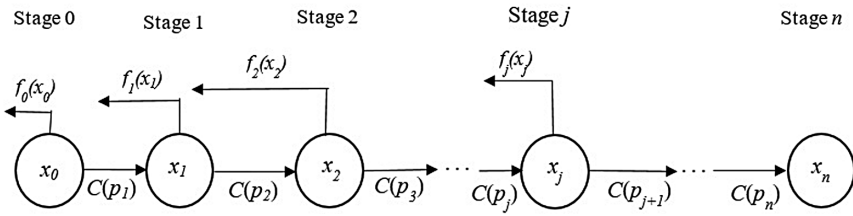


Figure 1.50 Graphical representation of the forward procedure of dynamic programming

There are also backward procedures, where computations start from the last stage and then go backward toward the first stage. The equations for the backward procedure can be written in the following way:

$$f_n(x_n) = 0$$

$$f_j(x_j) = \min_{\text{feasible proposals } p_j} \{f_{j+1}(x_{j+1}) + C_j(p_j)\} \quad j = 1, \dots, n$$

Example 1.10

A public transit operator serves 7 bus routes with 89 buses. Depot and bus lines served by the depot are shown in Figure 1.51.

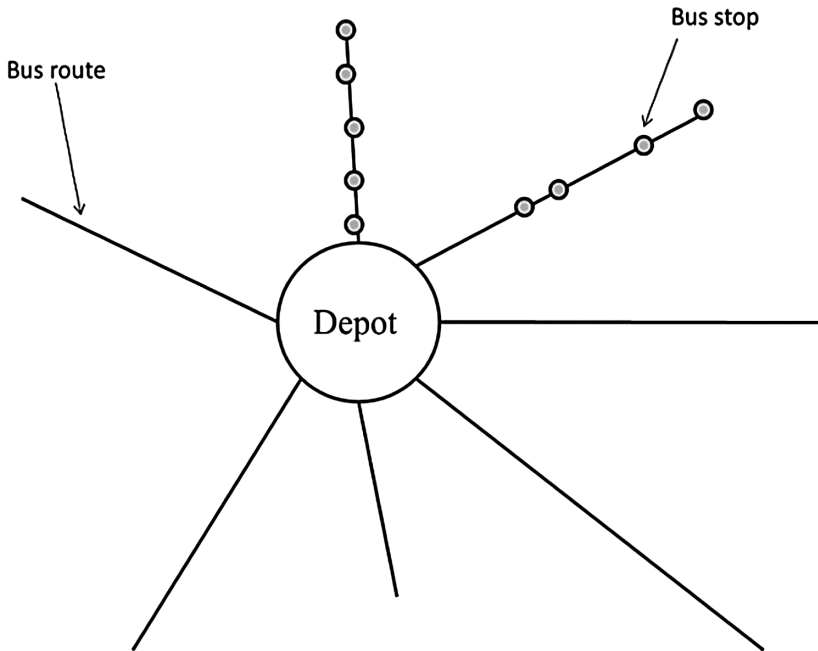


Figure 1.51 Depot and bus routes served by the depot

Characteristics of the routes are shown in Table 1.23. The capacity of one C_b bus is $C_b = 100$.

where:

- λ_i [Pass./h] – passenger arrival rate at route i
- q_i^{max} [Pass./h] – maximum load section at route i
- T_i [h] – bus turnaround time at route i (an elapsed interval of time in which a bus can pass line i in both directions)
- N_i – planned number of buses at route i
- C_i [Pass./h] – the offered capacity at route i
- α_i – load factor at route i
- $N_{i,min}$ – minimal number of buses necessary
- h_i^{max} – maximum allowed headway at route i

A certain number of the planned buses could, from time to time, be out of operation due to various technical reasons. A smaller number of buses in operation than is planned could also be caused by driver absenteeism. Suppose that 15 buses are out of operation. Assign the remaining 74 buses on the routes in a way to minimize the total waiting time of passengers. The time interval over which we consider the problem is $T^o = 1$ h.

SOLUTION:

The time interval over which we consider the problem is $T^o = 1$ h. Let us note the i -th bus route (Figure 1.52).

We denote by q_i^{max} the number of carried passengers per hour on the maximum load section of bus route i . We also denote by M_i the assigned number of buses on route i . The service frequency f_i represents the number of vehicles per time unit past a specific point in the same direction. The frequency equals:

$$f_i = \frac{M_i}{T_i} \left[\frac{veh}{h} \right]$$

The frequency f_i is expressed as the number of vehicles per hour. Headway in public transportation operations represents the time interval between vehicles past a specific point. Headways are expressed in minutes. As the frequency represents the number of vehicles per time unit past a specific point in the same direction, the frequency is the inverse of the headway, i.e.:

$$f_i = \frac{1}{h_i}$$

It has been shown, and widely accepted, that, in the case of regular bus arrivals, the average waiting time per passenger, w_i , represents one half of a headway, i.e.:

$$w_i = \frac{1}{2} \cdot h_i = \frac{1}{2} \cdot \frac{1}{f_i} = \frac{T_i}{2M_i}$$

Table 1.23 Bus route characteristics

Bus route	λ_i [Pass./h]	q_i^{max} [Pass./h]	T_i [h]	N_i	f_i [Departures/h]	h_i [h]	C_i [seats/h]	Load factor α_i	$N_{i,min}$	h_i^{max} [h]
1	1500	300	1	5	5	0.2	500	0.6	3	0.25
2	7000	1500	1.5	30	30	0.05	2000	0.75	23	0.1
3	3500	800	1	10	10	0.1	500	0.8	8	0.2
4	2000	400	1	5	5	0.2	500	0.8	4	0.2
5	6000	1400	1	20	20	0.05	2000	0.7	14	0.167
6	3000	600	1.5	10	15	0.1	1000	0.6	9	0.2
7	1000	200	1	4	4	0.25	400	0.5	2	0.333

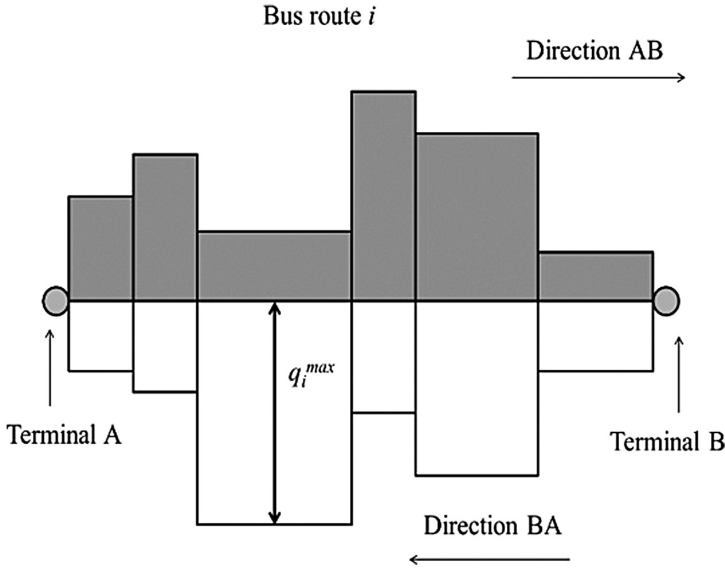


Figure 1.52 The number of passengers carried per hour on different load sections of bus route i

The total passenger waiting time T_w on all 7 bus routes during the time interval T^o in question equals:

$$T_w = \sum_{i=1}^7 \lambda_i \cdot T^o \cdot \frac{T_i}{2 \cdot M_i}$$

If we wish to transport all passengers on all seven bus routes, the following inequality must be satisfied:

$$q_i^{max} \cdot T_i \leq C_b \cdot M_i \quad \forall i = 1, 2, \dots, 7$$

i.e.:

$$M_i \geq \frac{q_i^{max} \cdot T_i}{C_b} \quad \forall i = 1, 2, \dots, 7$$

For every bus line i , we can also introduce the maximum allowed headway, h_i^{max} . The maximum allowed headway h_i^{max} on bus route i is one of the basic indicators of the level of service offered to the passengers. We will assign buses to the bus routes in such a way that no one bus route has headway greater than the maximum allowed headway. In a case of existing maximum allowed headways h_i^{max} for every bus route i , the following inequality must be satisfied:

$$\frac{T_i}{M_i} \leq b_i^{\max}$$

We try to assign available buses to bus routes in such a way as to minimize the total passenger waiting time on all bus routes. Since there is a shortage of buses, the assigned number of buses M_i at any bus route must be less than or equal to the planned number of buses, N_i . Mathematical formulation of our problem reads:

Minimize

$$T_w = \sum_{i=1}^7 \lambda_i \cdot T^o \cdot \frac{T_i}{2 \cdot M_i}$$

subject to:

$$\max \left\{ \frac{T_i}{b_i^{\max}}, \frac{q_i^o \cdot T_i}{C_b} \right\} \leq M_i \leq N_i \quad i = 1, 2, \dots, 7$$

$$\sum_{i=1}^7 M_i = 74$$

$$M_i \geq 0 \quad \text{and integer} \quad i = 1, 2, \dots, 7$$

The objective function, which represents the total waiting time of all passengers, should be minimized. First constraint defines how many buses we should assign to each route. The total number of available buses is 74. This is given by second constraint. Decision variables M_i are defined as integers by third constraint.

By taking into account first constraint, we calculate the following possible number of buses at specific routes:

$$4 \leq M_1 \leq 5$$

$$23 \leq M_2 \leq 30$$

$$8 \leq M_3 \leq 10$$

$$M_4 = 5$$

$$14 \leq M_5 \leq 20$$

$$9 \leq M_6 \leq 15$$

$$3 \leq M_7 \leq 4$$

We decide that stages $i = 1, 2, \dots, 7$ in our example are bus routes. We denote by 1, 2, ..., 7 the first, second, ..., seventh stages (bus routes).

State x_i at stage i represents the total number of buses assigned to the first, second, ..., and the j -th bus route, i.e.:

x_1 – number of buses assigned to the first route

x_2 – number of buses assigned to the first and second routes

x_3 – number of buses assigned to the first, second, and third routes

$\vdots$

x_7 – number of buses assigned to the first, second, third, fourth, fifth, sixth, and the seventh routes

In other words:

$$x_i = M_1 + M_2 + \cdots + M_i, \quad i = 1, 2, \dots, 7$$

The values of the $x_1, x_2, x_3, x_4, x_5, x_6, x_7$ must be within the following intervals:

x_1 :

$$4 \leq x_1 \leq \min\{5, 74 - (23 + 8 + 5 + 14 + 9 + 3)\} = \min\{5, 12\}$$

$$4 \leq x_1 \leq 5$$

x_2 :

$$4 + 23 \leq x_2 \leq \min\{5 + 30, 74 - (8 + 5 + 14 + 9 + 3)\} = \min\{35, 35\}$$

$$27 \leq x_2 \leq 35$$

x_3 :

$$27 + 8 \leq x_3 \leq \min\{35 + 10, 74 - (5 + 14 + 9 + 3)\} = \min\{45, 43\}$$

$$35 \leq x_3 \leq 43$$

x_4 :

$$35 + 5 \leq x_4 \leq \min\{43 + 5, 74 - (14 + 9 + 3)\} = \min\{48, 48\}$$

$$40 \leq x_4 \leq 48$$

x_5 :

$$40 + 14 \leq x_5 \leq \min\{48 + 20, 74 - (9 + 3)\} = \min\{68, 62\}$$

$$54 \leq x_5 \leq 62$$

x_6 :

$$54 + 9 \leq x_6 \leq \min\{62 + 15, 74 - 3\} = \min\{77, 71\}$$

$$63 \leq x_6 \leq 71$$

The x_7 must be equal to 74, i.e.:

$$x_7 = 74$$

Let us calculate the total waiting time if we assign buses 4 and 5 to bus route 1. The waiting times are:

$$f_1(4) = \lambda_1 \cdot T^o \frac{T_1}{2 \cdot M_1} = 1500 \cdot 1 \cdot \frac{1}{2 \cdot 4} = 187.5$$

$$f_1(5) = \lambda_1 \cdot T^o \frac{T_1}{2 \cdot M_1} = 1500 \cdot 1 \cdot \frac{1}{2 \cdot 5} = 150$$

These values are shown in Table 1.24. From the table, we can conclude that optimal values for M_1 , if x_1 are 4 and 5, are 4 and 5, respectively.

The optimal values of M_2 , for different values of x_2 , are given in Table 1.25.

Let us clarify the performed calculations. The total waiting time, if we assign 27 buses to bus routes 1 and 2, is:

$$f_2(27) = \lambda_2 \cdot T^o \frac{T_2}{2 \cdot M_2} + f_1(4) = 7000 \cdot 1 \cdot \frac{1.5}{2 \cdot 23} = 228.26 + 187.5 = 415.76$$

If we assign 28 buses to bus routes 1 and 2, the total waiting time equals:

$$\begin{aligned} f_2(28) &= \min_{M_2=23,24} \left\{ \lambda_2 \cdot T^o \frac{T_2}{2 \cdot M_2} + f_1(28 - M_2) \right\} \\ &= \min_{M_2=23,24} \left\{ 7000 \cdot 1 \cdot \frac{1.5}{2 \cdot M_2} + f_1(28 - M_2) \right\} \end{aligned}$$

$$\begin{aligned} f_2(28) &= \min_{M_2=23,24} \left\{ \frac{5250}{M_2} + f_1(28 - M_2) \right\} \\ &= \min \begin{cases} 228.26 + f_1(5) = 378.26 & \text{for } M_2 = 23 \\ 218.75 + f_1(4) = 406.25 & \text{for } M_2 = 24 \end{cases} = 378.26 \Rightarrow M_2^* = 23 \end{aligned}$$

The waiting time of 378.26 hours is obtained for $M_2^* = 23$.

Table 1.24 Optimal values of M_1 for different values of x_1

x_1	$M_1 = 4$	$M_1 = 5$	$f_1(x_1)$	M_1^*
4	187.5	—	187.5	4
5	—	150	150	5

Table 1.25 Optimal values of M_2 for different values of x_2

x_2	M_2								$f_2(x_2)$	M_2^*
	23	24	25	26	27	28	29	30		
27	415.76	—	—	—	—	—	—	—	415.76	23
28	378.26	406.25	—	—	—	—	—	—	378.26	23
29	—	368.75	397.5	—	—	—	—	—	368.75	24
30	—	—	360	389.42	—	—	—	—	360	25
31	—	—	—	351.92	381.94	—	—	—	351.92	26
32	—	—	—	—	344.44	375	—	—	344.44	27
33	—	—	—	—	—	337.5	368.53	—	337.5	28
34	—	—	—	—	—	—	331.03	362.5	331.03	29
35	—	—	—	—	—	—	—	325	325	30

If we assign 29 buses to bus routes 1 and 2, the total waiting time equals:

$$\begin{aligned}
 f_2(29) &= \min_{M_2=24,25} \left\{ \frac{5250}{M_2} + f_1(29 - M_2) \right\} \\
 &= \min \begin{cases} 218.75 + f_1(5) = 368.75 & \text{for } M_2 = 24 \\ 210 + f_1(4) = 397.5 & \text{for } M_2 = 25 \end{cases} \\
 &= 368.75 \Rightarrow M_2^* = 24
 \end{aligned}$$

The waiting time of 368.75 h is obtained for $M_2^* = 24$.

We calculated waiting times for $x_2=30$, $x_2=31$, $x_2=32$, $x_2=33$, $x_2=34$, and $x_2=35$ in the same way.

The results obtained are given in Table 1.25. For example, we see from Table 1.25 that, for $x_2=27$, the value of M_2 could only be equal to 23. Consequently, $f_2(x_2)$ equals 415.5 and M_2^* is 23. In the case where $x_2=28$, the M_2 can take values 23 and 24. The minimum of 378.26 and 406.25 is 378.26. In other words, since $f_2(x_2) = \min\{378.26, 406.25\} = 378.26$, the optimal value for M_2^* is

23. We fill the table for the other values of x_2 in the same way.

Table 1.26 Optimal values of M_3 for different values of x_3

x_3	$M_3=8$	$M_3=9$	$M_3=10$	$f_3(x_3)$	M_3^*
35	634.511	—	—	634.511	8
36	597.011	610.205	—	597.011	8
37	587.5	572.705	590.761	572.705	9
38	578.75	563.194	553.261	553.261	10
39	570.673	554.444	543.75	543.75	10
40	563.194	546.368	535	535	10
41	556.25	538.889	526.923	526.923	10
42	549.784	531.944	519.444	519.444	10
43	543.75	525.479	512.5	512.5	10

Table 1.27 The optimal values of M_4 for different values of x_4

x_4	$M_4 = 5$	$f_4(x_4)$	M_4^*
40	834.511	834.25	5
41	797.011	796.75	5
42	772.705	772.44	5
43	753.261	753	5
44	743.75	743.75	5
45	735	735	5
46	726.923	726.92	5
47	719.444	719.44	5
48	712.5	712.5	5

Table 1.28 The optimal values of M_5 for different values of x_5

x_5	M_5							$f_5(x_5)$	M_5^*
	14	15	16	17	18	19	20		
54	1048.54	—	—	—	—	—	—	1048.54	14
55	1011.04	1034.25	—	—	—	—	—	1011.04	14
56	986.73	996.75	1021.75	—	—	—	—	986.73	14
57	967.29	972.44	984.25	1010.72	—	—	—	967.29	14
58	958.04	953.00	959.94	973.22	1000.92	—	—	953.00	15
59	949.29	943.75	940.50	948.91	963.42	992.14	—	940.50	16
60	941.21	935.00	931.25	929.47	939.11	954.64	984.25	929.47	17
61	933.73	926.92	922.50	920.22	919.67	930.33	946.75	919.67	18
62	926.79	919.44	914.42	911.47	910.42	910.89	922.44	910.42	18

The optimal values of M_3 , for different values of x_3 , are given in Table 1.26.

The optimal values of M_4 , for different values of x_4 , are given in Table 1.27.

The optimal values of M_5 , for different values of x_5 , are given in Table 1.28.

The optimal values of M_6 , for different values of x_6 , are given in Table 1.29.

The optimal values of M_7 , for different values of x_7 , are given in Table 1.30.

We can conclude from Table 1.30 that $M_7^* = 4$. When we assign 4 buses to bus route 7, it leaves $74 - 4 = 70$ buses for other routes. We see from Table 1.30 that, for $x_6 = 70$, M_6^* equals 13 ($M_6^* = 13$). When we assign 13 buses to route 6, it leaves $70 - 13 = 57$ buses for the first 5 routes. We make conclusions about the optimal number of buses assigned to specific bus routes from the remaining tables. The optimal numbers of buses that should be assigned to specific bus routes are given in Table 1.31.

The graphical representation of the optimal solution is given in Figure 1.53.

Table 1.29 The optimal values of M_6 for different values of x_6

x_6	M_6							$f_6(x_6)$	M_6^*
	9	10	11	12	13	14	15		
63	1298.54							1298.54	9
64	1261.04	1273.54	—	—	—	—	—	1261.04	9
65	1236.73	1236.04	1253.08	—	—	—	—	1236.04	10
66	1217.29	1211.73	1215.58	1236.04	—	—	—	1211.73	10
67	1203.00	1192.29	1191.27	1198.54	1221.61	—	—	1191.27	11
68	1190.50	1178.00	1171.83	1174.23	1184.11	1209.25	—	1171.83	11
69	1179.47	1165.50	1157.55	1171.83	1159.80	1171.75	1198.54	1157.55	11
70	1169.67	1154.47	1145.05	1140.50	1140.36	1147.44	1161.04	1140.36	13
71	1160.42	1144.67	1134.02	1128.00	1126.08	1128.00	1136.73	1126.08	13

Table 1.30 The optimal values of M_7 , for different values of x_7

x_7	$M_7=3$	$M_7=4$	$f_7(x_7)$	M_7^*
74	1292.74	1265.36	1265.36	4

Table 1.31 The optimal numbers of buses that should be assigned to specific bus routes

Bus route	Number of buses to be assigned to the bus route
1	$M_1^* = 5$
2	$M_2^* = 23$
3	$M_3^* = 10$
4	$M_4^* = 5$
5	$M_5^* = 14$
6	$M_6^* = 13$
7	$M_7^* = 4$

PROBLEMS

- 1.1. A port operator should load at least 2000 t of goods to the ship in 8 h. Two cranes should be used for loading. The productivity of the first crane is 200 t/h, and of the second one 170 t/h. The cost of using the first crane is \$40/h and the cost of using the second crane is \$30/h. The cranes can work simultaneously. Determine the optimal solution (the amount of goods loaded by each crane) in such a way as to minimize the total unloading cost (make and solve the linear program).

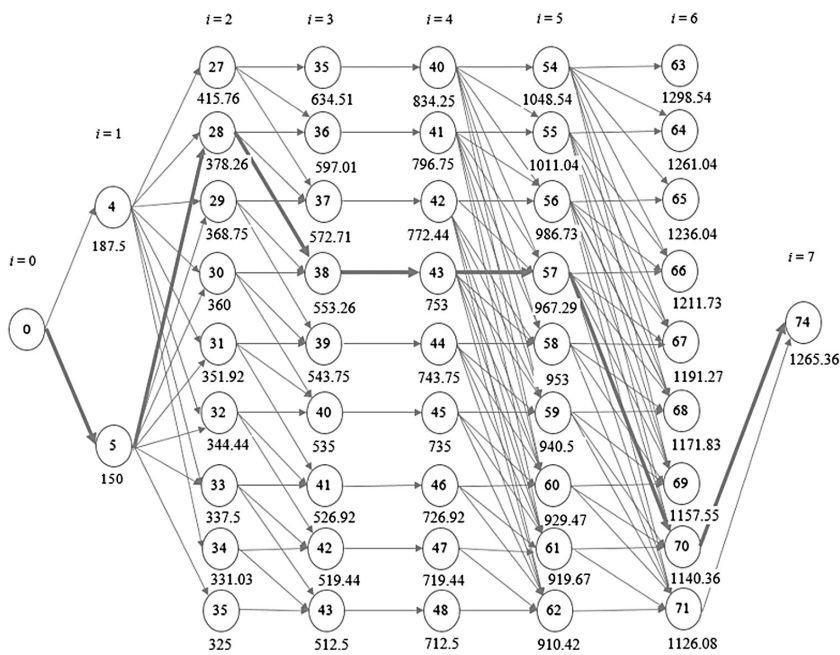


Figure 1.53 Optimal solution of the bus allocation problem

- 1.2. Four supermarkets (S1, S2, S3 and S4) need one type of goods. They have the following demands: 3, 5, 8, and 6 tons of goods. There are three warehouses (W1, W2 and W3) that supply the supermarkets. The warehouses have the following quantity of goods: 7, 12, and 9 tons of goods. The unit transportation costs from each warehouse to each supermarket are given in Table 1.32. Determine the optimal plan of transportation to minimize the total transportation costs.
- 1.3. There are 4 ships and 5 berths. Ships 1 and 2 could be served at berths 1, 2, and 3. Ships 3 and 4 could be served at any berth. One berth can serve just one ship and one ship can be served at only one berth. Ships' service times (in hours) are given in Table 1.33. Create an integer problem to minimize the total waiting time.
- 1.4. Two distribution centers should be opened in the transportation network given in Figure 1.54. Distribution centers could be located at

Table 1.32 Transportation costs (\$/t)

	S1	S2	S3	S4
W1	30	15	25	20
W2	25	23	18	30
W3	20	20	27	25

Table 1.33 Ship service time at the berths (in h)

	Berth 1	Berth 2	Berth 3	Berth 4	Berth 5
Ship 1	10	12	13	—	—
Ship 2	9	10	11	—	—
Ship 3	7	9	10	12	14
Ship 4	8	10	12	15	17

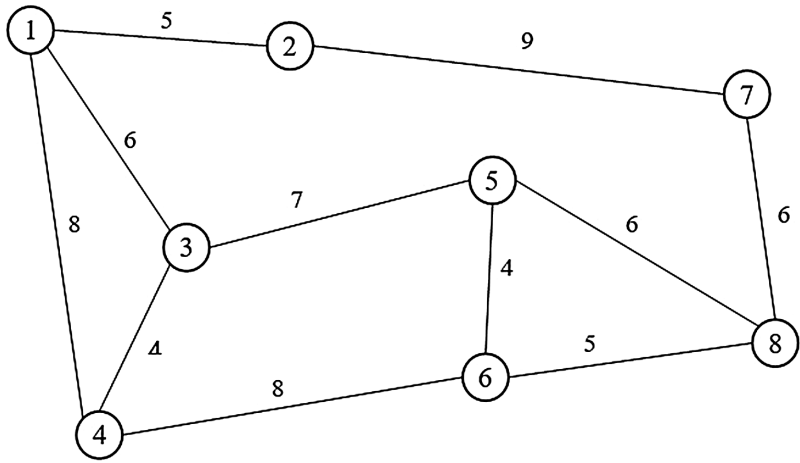


Figure 1.54 Transportation network

any node of the network. The nodes need the following number of deliveries per day: node 1 – 10, node 2 – 8, node 3 – 15, node 4 – 12, node 5 – 18, node 6 – 5, node 7 – 15, and node 8 – 20.

Determine the nodes where the distribution should be located and allocate nodes to distribution centers in the way to minimize the total transportation cost.

- 1.5. One hundred pallets should be loaded onto trucks within 8 h. Three forklifts could be used for loading. The following are the loading times of the forklifts: 5 min per pallet (for the first forklift), 8 min per pallet (for the second forklift) and 10 min per pallet (for the third forklift). The fixed cost of using a forklift is the same for each forklift and is equal to \$50 (this cost should be taken into account if the forklift works). The variable costs for forklifts 1, 2, and 3 are \$4, \$3, and \$2 per pallet, respectively. Determine the number of pallets loaded by each forklift to minimize the total cost. Write the integer linear program, the goal of which is to minimize the total cost.

BIBLIOGRAPHY

- Bazaraa, M. S., Jarvis, J. J., and Sherali, H. D., *Linear Programming and Network Flows*, 4th ed., John Wiley and Sons, Inc., Hoboken, NJ, 2010.
- Bellman, R., The Theory of Dynamic Programming, *Bulletin of the American Mathematical Society*, 60, 503–516, 1954.
- Bellman, R. (1957), *Dynamic Programming*, Princeton University Press. Dover paperback edition, 2003.
- Bellman, R. and Dreyfus, S., *Applied Dynamic Programming*, The RAND Corporation, Santa Monica, CA, 1962.
- Bertsimas, D. and Tsitsiklis, J. N., *Introduction to Linear Optimization*, Athena Scientific, Nashua, NH, 1997.
- Ceder, A., *Public Transit Planning and Operation: Modeling, Practice and Behavior*, 2nd ed., CRC Press, Taylor & Francis Group, 2015.
- Chen, D.-S., Batson, R. G., and Dang, Y., *Applied Integer Programming: Modeling and Solution*, John Wiley and Sons, Hoboken, NJ, 2010.
- Cordeau, J. F., Laporte, G., Savelsbergh, M. W. P., and Vigo, D., Vehicle Routing. In C. Barnhart and G. Laporte (Eds.), *Transportation*, Elsevier, Amsterdam, 2007.
- Dantzig, G. B., *Linear Programming and Extensions*, Princeton University Press, Princeton, NJ, 1963.
- Dantzig, G. B. and Thapa, M. N., *Linear Programming 1: Introduction*, Springer Verlag, New York, 1997.
- Dantzig, G. B. and Thapa, M. N., *Linear Programming 2: Theory and Extensions*, Springer Verlag, New York, 2003.
- Desrochers, M., Lenstra, J. K., Savelsbergh, M. W. P. and Soumis, F., Vehicle Routing with Time Windows: Optimization and Approximation. In: B. L. Golden and A. A. Assad (Eds.), *Vehicle Routing: Methods and Studies*, North-Holland, Amsterdam, 65–84, 1988.
- Gass, S. I., *Linear Programming: Methods and Applications*, 5th ed., McGraw-Hill, New York, 1985.
- Hillier, F. and Lieberman, G., *Introduction to Operations Research*, McGraw - Hill Publishing Company, New York, 1990.
- Karlot, J. K., *Integer Programming: Theory and Practice*, CRC Press, Boca Raton, FL, 2006.
- Kaufmann, A., *Graphs, Dynamic Programming, and Finite Games*, Academic Press, New York-London, 1967.
- Lee, D. H., Chen, J. H., and Cao, J. X., The Continuous Berth Allocation Problem: A Greedy Randomized Adaptive Search Solution, *Transportation Research Part E*, 46, 1017–1029, 2010.
- Little, J. D. C., Murty, K. G., Sweeney, D.W., and Karel, C., An Algorithm for the Travelling-Salesman Problem, *Operations Research*, 11, 6, 972–989, 1963.
- Luenberger, D. and Yinyu, Y., *Introduction to Linear and Nonlinear Programming*, 3rd ed., International Series in Operations Research and Management Science, Volume 116. Springer, New York, 2008.
- Nemhauser, G. L. and Wolsey, L. A., *Integer and Combinatorial Optimization*, Wiley Interscience, New York, 1988.
- Potts, R. B. and Oliver, R. M., *Flows in Transportation Networks*, Academic Press, New York, 1972.

- Schrijver, A., *Theory of Linear and Integer Programming*, John Wiley and Sons, New York, 1998.
- Sierksma, G. and Zwols, Y., *Linear and Integer Optimization: Theory and Practice*, 3rd ed., CRC Press, Boca Raton, FL, 2015.
- Teodorović, D. and Rallis, T., A Model for Assigning Vehicles to Scheduled Routes When There Is a Shortage of Vehicles, *Transportation Planning and Technology*, 12, 135–150, 1988.
- Teodorović, D. and Vukadinović, K., Logistics Engineering Tool Chest. In G. Don Taylor (ed.), *Logistics Engineering Handbook*, 3–1- 3-30, CRC Press, Boca Raton-London-New York, 2008.
- Taha, H., *Operations Research*, MacMillan Publishing Co., Inc., New York, 1982.
- Winston, W., *Operations Research*, Duxbury Press, Belmont, CA, 1994.
- Wolsey, L. A., *Integer Programming*, John Wiley and Sons, New York, 1998.